

Research Report on

Empirical methods for rationale-first LLM prompting in vulnerability detection and code generation

Full search query: I want to find empirical and methodological research on rationale-first / explain-before-answer prompting with large language models for software engineering tasks—especially vulnerability detection and code generation—that investigates how requiring an explanation before the main output affects explanation quality (using any explicit operationalization of usefulness and/or faithfulness) and task performance

Summary

Rationale first / explain before answer prompting is now common in LLM based vulnerability detection and repair, and there is strong evidence that well structured rationale first pipelines improve patching performance [7 10], but only a small subset of work actually measures explanation quality in a principled way or cleanly isolates how explanation first prompting affects both rationale quality and task metrics [1 4,6,12,15].

Overall answer to the search goal

- **You did find directly relevant work, but it is fragmented along two axes:**
 - For **vulnerability detection**, many papers compare CoT / rationale producing prompts with direct answers, and a few explicitly score explanation quality, but almost none isolate a strict “explain before answer vs answer before explain vs no explanation” contrast under tight controls [1 4,6,12,15].
 - For **vulnerability repair/patching**, there is clear evidence that pipelines which force the model to do explicit, structured reasoning (root cause / strategy / patch) before emitting code significantly improve patch success [7 10], yet these papers almost never evaluate the correctness, faithfulness, or human usefulness of the rationales themselves.
- **Net effect:** we have good indications that rationale first prompting can help security critical code generation, mixed and often negative results for generic CoT in detection, and only early, methodologically uneven attempts to quantify explanation usefulness/faithfulness in this domain.

Rationale first prompting for vulnerability detection: what we know

Prompting styles and performance

- Across major detection evaluations, **CoT or explanation producing prompts are routinely included** but their effects are mixed and highly dependent on structure:
 - SecLLMHolmes shows that “Step by Step” (security expert style CoT) reasoning is brittle—highly sensitive to minor code changes (renaming, whitespace, libraries)—and that explanations are often wrong even when the vulnerability label is correct [1].
 - Khare et al. find that a CoT “let’s think step by step” prompt does not outperform a strong dataflow style prompt across 16 models and 5 datasets; few shot examples also add limited value [15].
 - Liu et al. and Yang et al. show that adding code specific information (similar code, DFGs, primitive API abstractions) plus CoT yields better accuracy than naive prompts, but the main gains often come from the structured context rather than from generic CoT wording alone [11,16].
- More structured reasoning prompts do better:
 - Real VulLLM’s decomposition and Plan and Solve prompts (which explicitly break the task into sub analyses before the verdict) generally outperform standard and plain CoT prompts, especially with GPT 4 [3].
 - VulInstruct uses specification guided reasoning—inferring security specifications from prior cases and applying them to new code—and reports large F1/recall gains when requiring both a correct prediction and “valid reasoning” [5].
 - LLMVulExp uses CoT style “key code extraction” and instruction tuning to jointly detect vulnerabilities and explain them, and reports improved detection over base models; however, the individual contribution of the CoT style prompting vs fine tuning is not disentangled [6].

High level takeaway for detection:

- Simply saying “think step by step” rarely helps and can even hurt (brittleness, over verbose, or degraded performance on long contexts) [1,13 15].
- Rationale first prompts that are **structured around domain concepts**—dataflow, security specifications, decomposition of sub tasks—can improve detection, but they have not been systematically compared against answer first or hidden rationale variants under tight experimental control [3,5,6,11].

Explicit explain before answer patterns

- Several detection oriented works use EBA like patterns (analyze then decide) in their prompts:
 - Yang et al. show CoT prompts that first ask for detailed code analysis (e.g., pointer operations) and only then a vulnerability decision plus reasons—an explicit explain then answer pattern [11].
 - Khare et al.'s CoT and “dataflow + self reflection” prompts require analysis and self evaluation before the final verdict, effectively a multi stage EBA protocol [15].
 - LLM4Vuln always elicits structured outputs with a label plus rationale and compares prompt schemes, including a “Post CoT” where reasoning is requested after an initial decision; this is the only detection paper in the set that explicitly treats reasoning timing (pre vs post decision) as a variable [12].
- However, none of these establish a clean, within model causal comparison between:
 - Explanation first,
 - Answer first,
 - Hidden reasoning (scratchpad not shown), while controlling for token budget and temperature. This is exactly the kind of methodological experiment your background calls for, and it is still largely missing.

Rationale first prompting for vulnerability repair and patching: stronger evidence on task performance

This is where rationale first prompting has the clearest, positive impact—though explanation quality itself is mostly unmeasured.

Two stage and multi step rationale first repair pipelines

- VRpilot (Kulsum et al.) is the clearest example of an explicit rationale first protocol for patching [8]:
 - Stage 1: a CoT prompt extracts a structured rationale (explanation of the vulnerability, its root cause, and potential fixes).
 - Stage 2: a second prompt uses this rationale to guide patch generation.
 - External feedback (compiler, sanitizers, tests) is then used to refine patches iteratively.
 - Ablation shows:
 - Removing the reasoning stage significantly reduces repair success,
 - Removing validation feedback also hurts, but both components are critical.
 - Repair performance improves by +14% correct patches for C and +7.6% for Java over baselines [8].
- APPATCH / LLMPATCH builds few shot exemplars that themselves have three steps—root cause analysis, fixing strategy, patch generation—and uses these to guide the model; this is essentially a baked in rationale first template [9]:
 - Large F1/recall gains are reported on real world vulnerabilities and Big Vul, including successful automatic fixes of zero day vulnerabilities [9].
 - A preliminary study suggests that providing this structured reasoning in exemplars aids patch generation, but they do not isolate “with vs without reasoning” in a controlled ablation like VRpilot.
- VSP (vulnerability semantics guided prompting) generalizes the approach to multiple tasks [10]:
 - For patching, prompts enforce a two step reasoning structure—first “Root cause analysis”, then “Patching strategy” before the patch.
 - Carefully designed exemplars focusing on “vulnerability semantics” significantly improve identification, discovery, and patching F1/accuracy compared with generic baselines.
 - They also report a negative result: a naive line by line CoT style did not work well, indicating that more reasoning tokens and finer granularity do not automatically help [10].

Multi agent rationale pipelines

- VulnPatch (Sakib and Khan) uses separate LLM agents for detection, explanation, and patching of smart contract vulnerabilities [7]:
 - They find that this decomposition—having an explicit explanation agent in the middle—performs better than a single agent handling all tasks.
 - They report 93.3% accuracy/F1, explanations for 84.44% of vulnerabilities, “90% sound reasoning” in explanations, and 84% patch validity [7].
 - The evaluation of “sound reasoning” is reported as a single percentage; the underlying rubric (e.g., human vs LLM judge, criteria) is not detailed in the excerpt.

High level takeaway for repair:

- In contrast to detection, **there is strong empirical evidence that structured rationale first prompting improves patch correctness**—especially when combined with:
 - External feedback (VRpilot) [8],
 - Carefully designed multi step exemplars (APPATCH, VSP) [9,10],
 - Decomposed agent roles (VulnPatch) [7].
 - Yet, these works treat rationales primarily as internal scaffolding to drive better code; they rarely score explanation quality or test whether “better” rationales (however defined) are causally linked to better patches.
-

Evaluation of explanation quality: usefulness and faithfulness

A central goal of your search was to find work that not only elicits explanations but also evaluates their usefulness and faithfulness in a security context. Such work exists, but it is limited and uneven.

Works that explicitly evaluate explanation quality

- SecLLMHolmes (Ullah et al.) [1]:
 - Introduces a “faithful reasoning” dimension and perturbation style robustness tests (renaming, whitespace, library changes).
 - Shows that reasoning is often incorrect or unfaithful even when the model correctly flags a vulnerability; CoT is brittle to small code augmentations.
 - This is a coarse but conceptually aligned form of behavioral faithfulness testing.
- Steenhoek et al. [2]:
 - Manually analyze 287 explanations and categorize errors into Code Understanding, Hallucination, Logic, and Common-sense.
 - Also track whether explanations correctly report vulnerability type, location, and root cause; models only correctly localized 6/27 bugs on DbgBench and 57% of explanations contained errors [2].
 - This is one of the most detailed error taxonomies of vulnerability explanations, directly addressing explanation usefulness to analysts.
- Real VulLLM [3]:
 - Evaluates LLMs on real CVEs using several reasoning oriented prompts (Standard, CoT, Decomposition, Plan and Solve).
 - Introduces a combined scoring measure that jointly reflects classification accuracy and “reasoning quality.”
 - They report that reasoning is frequently incomplete or incorrect even when the classification is right, especially for unfamiliar vulnerability patterns [3].
- CORRECT [4]:
 - Perhaps the most structurally aligned with your interests:
 - Uses context rich prompts to get CoT rationales for both vulnerable and patched versions of a function.
 - Uses a second LLM as judge prompt that compares rationales against ground truth evidence (CVE text, CWE, commit message, code diff) to evaluate whether the explanation captures the root cause for vulnerable versions and avoids false alarms for patched versions.
 - This implements an explicit, contrastive notion of explanation faithfulness grounded in real world evidence.
- LLMVulExp [6]:
 - Instruction tunes models to generate both detection and explanations.
 - Proposes a 3 dimensional evaluation of explanations—accuracy, clarity, actionability—using LLM based scoring with expert verification.
 - This makes “developer usefulness” of explanations an explicit target metric, going beyond just correctness.
- VulnInstruct [5]:
 - Requires “valid reasoning” along with correct predictions and conducts manual failure analysis (e.g., zero reasoning, miscategorization, incomplete mechanism understanding).
 - While the exact scoring of “valid reasoning” is not fully formalized, this is one of the few works where a prediction does not count as correct unless the reasoning meets certain criteria.
- VulnPatch [7]:
 - Reports that explanations exhibit “90% sound reasoning,” implying some evaluation of reasoning quality.
 - The excerpt does not specify how this is assessed; nonetheless, it is a rare case where explanation quality is reported alongside patch validity.

Across these, the most directly relevant for explicit and structured explanation quality are:

- **Faithfulness oriented:** SecLLMHolmes [1], CORRECT [4], Real VulLLM [3].
- **Usefulness oriented (developer centric)** LLMVulExp [6], Steenhoek et al.'s error categories [2], and to a lesser extent VulInstruct [5].

Limitations relative to your desired operationalizations

Compared to the rich faithfulness/usefulness criteria you outlined, current work has notable gaps:

- Faithfulness:
 - Some perturbation and robustness testing exists (e.g., code augmentations in [1]), but there are **no systematic counterfactual experiments** like:
 - Modifying only the lines/variables that the explanation claims are critical and checking whether the model's decision changes as predicted.
 - Localization based faithfulness is only indirectly addressed:
 - Steenhoek et al. report poor localization and explanation errors [2],
 - LOVA uses attention maps for line level localization but does not link explanations to those lines in a faithfulness analysis [13,14].
- Usefulness:
 - LLMVulExp's actionability metric is the closest to directly operationalizing "does this explanation help a developer fix the issue?" [6].
 - CORRECT and Real VulLLM implicitly tie explanation correctness to usefulness via contrastive evaluation and combined scores [3,4].
 - No paper in this set implements a controlled human in the loop study where developers receive explanations vs no explanations and their performance (bug finding, patching time, patch quality) is measured.
- Ordering effects (explain before vs answer before vs hidden):
 - LLM4Vuln is the only work that explicitly manipulates timing (Post CoT vs other schemes) [12], but it focuses on overall "vulnerability reasoning" capability rather than a rigorous comparison of explanation first vs answer first at fixed token budgets.
 - There is no study that, for the same model and samples, systematically compares:
 - Direct answer,
 - Rationale first,
 - Answer first (post hoc explanation),
 - Hidden rationales, and jointly measures both task performance and explanation quality.

Vulnerability localization: CoT as a baseline, attention and multi trajectory as alternatives

- LOVA related papers [13,14] are relevant to your interest in explanation linked localization:
 - They evaluate four regimes: direct output, few shot CoT (explicit rationale first), Mixture of Agents, and rStar (multiple reasoning trajectories with consistency based selection).
 - Findings:
 - CoT is often not the best regime, especially on long code contexts, where degradation and overthinking effects appear [13,14].
 - Simple line by line checking guided by attention maps can substantially improve localization performance over CoT [14].
- However, they:
 - Do not explicitly score rationales for correctness or faithfulness.
 - Use attention and line level predictions as "explanations" more than natural language rationales.
- From the viewpoint of your research agenda, these papers mainly show:
 - CoT/rationale first is not a panacea for localization.
 - Alternative mechanisms (attention based heuristics, multiple reasoning paths with consistency filtering) may be more robust and might be fruitfully combined with structured rationales in future work.

Methodological gaps and opportunities aligned with your goals

Given your original goal—understanding how requiring an explanation before the main output affects explanation quality and task performance in software security tasks—the current literature leaves several important gaps:

1. Causal isolation of explanation first effects:
 - No study cleanly compares explain before answer vs answer before vs hidden rationale vs direct answer for the same model, same items, same token budgets.
 - VRpilot and APPATCH show that adding a rationale first stage improves patching [8,9], but they don't compare to equivalent answer first pipelines with matched resources, nor analyze whether explanation quality mediates the effect.
1. Strong faithfulness tests:
 - Existing work rarely ties explanations to the model's decision process via counterfactual edits to code elements named in the rationale.
 - A natural extension of CORRECT, LLMVulExp, or LLM4Vuln would be to:
 - Use the rationale to identify claimed critical elements (e.g., source, sink, missing check),
 - Perturb only those, re query the model, and measure whether the decision changes appropriately.
1. Rich usefulness evaluations:
 - We lack controlled user studies where security engineers or developers perform tasks (vulnerability triage, localization, patching) with vs without model explanations.
 - LLMVulExp's actionability and Real VulLLM's combined scores are first steps [3,6], but they rely on LLM as judge or high level rubrics rather than end user performance.
1. Structured, domain aligned rationales for code:
 - The most promising performance results come from rationales structured around security abstractions (vulnerability semantics, security specifications, decomposition, plan and solve) [3,5,9,10].
 - Yet explanation evaluation remains largely unstructured. A key opportunity is to:
 - Design rationales with explicit slots (source, sink, sanitization, control flow path, assumption, patch strategy),
 - Score each slot against ground truth (e.g., via manual coding or LLM as judge calibrated to expert labels).
1. Long context and non local reasoning:
 - Localization and context rich frameworks show that long, non local vulnerabilities stress CoT and encourage hallucinations or incorrect assumptions [4,11,13,14].
 - Rationale first protocols need to account for:
 - Context budgeting (explanations compete with code tokens),
 - Non local dependencies (cross file, cross function),
 - Potential benefits of hybrid approaches (program analysis + structured rationales + attention based hints).

Practical guidance for your own work

Based on what was found, if you want to design rigorous experiments on rationale first prompting for vulnerability detection and code generation:

- Use existing frameworks and datasets that already expect rationales:
 - CORRECT for contrastive vulnerable/patched pairs and root cause centered rationales [4].
 - UniVul/LLM4Vuln for structured label+rationale outputs and prompt scheme comparisons [12].
 - LLMVulExp for explanation centric evaluation dimensions (accuracy, clarity, actionability) [6].
- For repair tasks, build on the strongest rationale first pipelines:
 - Start from VRpilot's two stage reasoning'patch protocol with external validation [8].
 - Incorporate VSP/APPATCH style vulnerability semantics rationales (root cause + strategy) [9,10].
 - Add explicit evaluation of rationale correctness (e.g., root cause alignment, localization) and test whether better scored rationales yield better patches.

- Explicitly manipulate reasoning order and visibility:
 - Implement controlled conditions: direct answer, rationale first, answer first, hidden rationale, each at the same total token budget and sampling scheme.
 - Measure both task metrics (accuracy, F1, patch success) and explanation metrics (faithfulness/usefulness) under each condition.
- Integrate faithfulness tests:
 - Use explanations to select lines/variables/edges to perturb; evaluate whether model predictions change as implied by its own rationale.
 - Compare with perturbations to equally sized random regions as a baseline.
- Consider hybrid designs:
 - Combine structured rationales (slots aligned to program analysis) with attention based hints or line by line checking [11,13,14].
 - Investigate whether rationales that correctly reference analysis derived elements (e.g., primitive APIs, taint paths) are more predictive of correct decisions or patches.

In short, the literature you found gives you strong evidence that rationale first prompting can improve security relevant code generation, early but promising frameworks for scoring explanations, and clear signs that naive CoT is insufficient—leaving significant room for more principled, causally informed studies of explanation first protocols in vulnerability detection and repair.

Categories

Comparative Overview of Rationale Oriented LLM Work in Vulnerability Tasks

High Level Landscape

Below is a compact comparison focused on aspects an expert would care about for rationale first / explain before answer prompting in software security tasks.

Key dimensions:

- Task focus and granularity (detection, localization, patching/repair).
- Prompting styles compared, especially any explicit explain before answer regimes.
- Whether explanation quality is explicitly evaluated (and how).
- Whether the paper analyzes the interaction between prompting/rationales and task performance.

Table 1 – Core Experimental Design and Prompting Schemes

(“EBA” = clear explain before answer, i.e., rationale first then decision/patch in the same prompt or pipeline.)

Ref	Main task(s)	Prompting schemes compared	Any explicit EBA protocol?	Notes on what is distinctive wrt this topic
[1]	Vulnerability detection (C, Python)	4 prompting techniques (zero/few shot × task/role oriented) and 3 styles: Standard (direct), Step by Step (CoT like), Definition based	Step by Step prompts emulate multi step security workflow; ordering of rationale vs verdict not clearly specified	Deep evaluation of “faithful reasoning” and robustness; shows that even when detection is correct, natural language reasoning is often wrong/unfaithful.
[2]	Vulnerability detection, type/location/root - cause prediction	Basic, In Context, CoT style (“CoT SA”), Contrastive Pairs	CoT style prompts that ask for explanations of predictions, but text does not clearly impose a strict “rationale block then answer” structure	Systematic prompt search; error taxonomy over 287 explanations (Code Understanding, Hallucination, Logic, Commonsense).
[3]	Vulnerability detection “in the wild” (real CVEs)	Standard, CoT (P - CoT), Decomposition (P Decomp), Plan and Solve (P P&S), each in ZS/FS	All three “reasoning” prompts explicitly elicit intermediate reasoning; ordering of reasoning and verdict not fully detailed	Real world CVE setting; joint score for detection accuracy and “reasoning quality”; compares multiple LLMs and prompt styles.
[4]	Vulnerability detection with contrastive vulnerable/patched pairs	Zero shot CoT “context rich” detection prompt; separate “rationale assessment” prompt	Prompt I: vulnerability assessment with CoT rationale + conclusion; Prompt II: LLM as judge scoring of those rationales	CORRECT framework: paired prompts on vulnerable vs patched versions; explicit requirement that rationales for vulnerable versions capture root cause.
[5]		Structured CoT style reasoning about specifica-	Uses structured reasoning steps, but EBA vs inter-	VulnInstruct: requires “valid reasoning” for a prediction

	Vulnerability detection guided by learned "security specifications"	tions; retrieval augmented prompts	leaving of reasoning/label not clearly specified	to count; uses structured specs as explanations.
[6]	Vulnerability detection with explanation generation	Prompt based annotation, then instruction tuning to jointly detect + explain; key code extraction via CoT	Explanations and verdict are co generated; not a strict EBA control condition	LLMVulExp: introduces an explicit 3 dimensional explanation evaluation (accuracy, clarity, actionability).
[7]	Smart contract vulnerability detection, explanation, and patching	Multi agent pipeline: detection agent ' explanation agent ' patching agent	Yes, at pipeline level: explanations are produced by a dedicated agent before patching	VulnPatch: fine tuned agents for each sub task; reports % of explanations judged to show "sound reasoning" and patch validity.
[8]	Vulnerability repair (C, Java)	VRpilot: 2 stage CoT pipeline (reasoning extraction step, then patch extraction step), plus patch - validation feedback	Yes: explicit rationale first CoT step whose output is then used to drive patching	Ablation shows both reasoning step and validation feedback are critical for improved repair success; no separate scoring of explanation quality.
[9]	Vulnerability patching (zero day and existing vulns)	CoT style few shot exemplars with 3 steps: (1) root cause analysis, (2) fixing strategy, (3) patch	Yes in exemplars: reasoning steps precede patch in examples; patch prompt follows the same three step structure	APPATCH: focuses on patch performance; preliminary evidence that reasoning helps patching, but explanation quality not quantitatively evaluated.
[10]	Vulnerability identification/discovery/patching	VSP: few shot CoT inspired exemplars emphasizing "vulnerability semantics", with 2 step patch prompt: root cause analysis ' patch strategy (and then patch)	Yes for patching: two step reasoning (root cause, strategy) before code patch	Carefully designed CoT exemplars; multiple baselines and ablations, but explanation quality not separately scored.
[11]	Vulnerability detection with program analysis derived context	Prompt strategies: CoT, few shot, in context; CoT examples ask model to summarize/analyze code (e.g., pointer operations) then decide vulnerable vs not and give reasons	Examples show explicit "first analyze, then decide" structure (EBA like); paper does not isolate this as a controlled condition	Focus is on how different context abstractions (primitive API summaries, etc.) and prompts affect detection performance.
[12]	Vulnerability detection and reasoning benchmarking	Multiple prompt schemes incl. CoT and "Post CoT" (CoT after initial decision), tool + knowledge variants	Both pre and post decision reasoning schemes; outputs structured as {label, rationale} via function calling	LLM4Vuln/UniVul: explicitly designed to decouple reasoning from external aids; uses GPT 4 to structure and evaluate rationales.
[13]	Vulnerability localization	4 decoding regimes: direct, few shot CoT (reasoning then answer), Mixture of Agents, rStar (multiple reasoning paths)	Yes: few shot CoT explicitly "first outputs reasoning process before final answer"	LOVA: compares localization accuracy across prompting/decoding regimes; CoT is a baseline, not the main contribution.
[14]	Vulnerability localization with line by line checking	Same 4 regimes as [13]	Yes, as in [13]	Emphasizes self attention + line by line checking to mitigate reasoning degradation; CoT EBA baseline included.
[15]	Vulnerability detection across 5 datasets and 16 LLMs	Dataflow style prompt, CoT ("let's think step by step"), and "dataflow + self reflection" (analysis ' self check ' verdict)	Yes: CoT and reflection prompts ask for reasoning chains and analysis before the final verdict	Finds CoT does not significantly outperform strong dataflow prompts; analyzes performance by CWE and prompt scheme.
[16]	Vulnerability detection with code specific info	Basic prompts, prompts with code specific info (similar code, DFG), CoT prompts; also varies position of code specific info and temperature	CoT prompts intended to "gradually detect" vulnerabilities, but ordering is not clearly described as a separate EBA regime	Exploration of prompt content and structure; focuses on detection performance effects.

Key takeaway:

- Many papers use CoT or multi step reasoning, but only a subset explicitly enforce and analyze explain before answer protocols in a way that cleanly separates rationale generation from final outputs. Particularly relevant EBA or EBA like setups include [4,7,8,9,10,11,13,14,15].

Comparative Treatment of Explanation Quality (Usefulness/Faithfulness)

Table 2 – How Explanations are Evaluated

Here, “usefulness” = any explicit measure of how explanations help tasks or users; “faithfulness” = alignment to ground truth causes/evidence or to the model’s decision basis.

Ref	Explanation explicitly evaluated?	Operationalization of explanation quality	Usefulness vs faithfulness emphasis	Notes
[1]	Yes, but at a high level	“Faithful reasoning” and robustness to code augmentations (renaming, whitespace, library changes); comparison of prompt styles; analysis that reasoning often incorrect even when detection correct	Primarily faithfulness (behavior under perturbations, qualitative reasoning correctness)	Shows CoT like reasoning is brittle; no fine grained localization or utility metrics in the excerpt.
[2]	Yes	Manual categorization of errors in 287 explanations into Code Understanding, Hallucination, Logic, Commonsense; also whether type/location/root cause are correctly reported	Mix of usefulness (does explanation convey correct type, location, cause) and faithfulness (how explanations reflect model’s apparent understanding)	One of the most detailed error taxonomies over LLM explanations in vulnerability detection.
[3]	Yes	Combined scoring measure that “jointly quantifies classification accuracy and reasoning quality”; qualitative observation that reasoning is incomplete/incorrect even when classification is right	Both usefulness and faithfulness (reasoning score is used alongside accuracy)	Exact scoring rubric not detailed in the excerpt; intended for real world deployment assessment.
[4]	Yes, explicitly and systematically	Prompt II uses LLM as a -judge with ground truth evidence (CVE text, CWE label, commit message, diff) to rate how well rationales capture root cause (and avoid false alarms on patched code)	Strong emphasis on faithfulness (alignment with known root cause) and a form of usefulness (contrastive correctness between vulnerable/patched)	CORRECT is the clearest example of structured rationales evaluated against ground truth evidence in this set.
[5]	Partially	“Strict criteria requiring both correct predictions and valid reasoning”; manual failure analysis of reasoning categories (“zero reasoning”, miscategorization, etc.)	Mostly faithfulness: reasoning must align with security specifications; usefulness is implicit via joint success criterion	Reasoning is structured around inferred “security specifications,” but detailed metrics are not in the excerpt.
[6]	Yes, with a defined protocol	3dimensional evaluation: accuracy, clarity, actionability; LLM based automatic scoring plus expert manual verification	Usefulness: clarity and actionability directly target developer utility; accuracy also touches faithfulness	LLMVulExp is the most explicit about explanation usability for practitioners.
[7]	Yes, but coarsely	Reports that explanations and patches show 90% “sound reasoning” and 84% patch validity	Both usefulness (supporting patching) and faithfulness (soundness of reasoning), but methodology not detailed in excerpt	Multi agent design allows reasoning quality to be evaluated per agent; criteria summarized as percentages.
[8]	Not directly (beyond anecdote)	No explicit scoring of rationales; ablation shows that having a CoT reasoning step improves repair success	Usefulness inferred indirectly from task performance (repair success); faithfulness not evaluated	VRpilot uses rationale - first prompting but treats reasoning as an internal mechanism rather than a measured artifact.
[9]	Not explicitly	Authors state reasoning “helpful to guide” correct patch generation; no quantitative rationale metrics in excerpt	Usefulness is argued qualitatively; faithfulness not measured	Focus is on patching metrics; reasoning steps are part of adaptive prompting.
[10]	Not explicitly	No separate rationale scoring; only task metrics (F1/accuracy, etc.)	Neither usefulness nor faithfulness is directly measured; rationales are instrumental	Provides careful CoT exemplar design but treats explanations as scaffolding.
[11]	Not explicitly	Explains reasoning patterns qualitatively; perfor-		

		mance compared across context/prompt variants	Usefulness/faithfulness not operationalized separately from detection metrics	CoT analyses are present but not graded.
[12]	Yes	Uses GPT 4 to annotate/structure outputs; compares prompt schemes incl. "Post CoT"; examines vulnerability reasoning as a capability	Primarily faithfulness (evaluating reasoning independent of tools/knowledge)	LLM4Vuln is explicitly about "vulnerability reasoning," but the excerpt doesn't detail the scoring rubric.
[13]	No explicit rationale scoring	Focus on localization accuracy; CoT/rStar/MoA are evaluated via performance metrics; attention maps visualized	Usefulness/faithfulness only indirectly via localization performance	Rationales are generated but not systematically evaluated.
[14]	No explicit rationale scoring	Similar to [13]; focuses on attention based localization improvements and prompt/decoding regimes	Same as [13]	CoT reasoning is a baseline; quality not graded.
[15]	Qualitative only	Compares prompt schemes including CoT and self reflection; qualitative inspection of reasoning chains reported	No formal usefulness/faithfulness metrics; only detection performance	Important for showing CoT does not always help over strong structured prompts.
[16]	No explicit rationale scoring	Focuses on detection performance while varying prompt content/structure and temperature	No direct explanation quality metrics	CoT used to "gradually detect" vulnerabilities; explanations not analyzed.

Key takeaways:

- Strong, explicit explanation quality evaluations in vulnerability settings appear in [1,2,3,4,6,7,12].
- Among those, [4] and [6] are the most structured for faithfulness/usefulness respectively:
 - [4] uses ground truth CVE/CWE/diff evidence and an LLM judge to score root cause coverage (faithfulness, including contrastive vulnerable vs patched analysis).
 - [6] defines accuracy, clarity, and actionability (a direct proxy for usefulness to developers).

Effects of Rationale First / CoT Prompting on Task Performance

Table 3 – Reported Impact of Reasoning Oriented Prompting on Security Task Metrics

This table aggregates how explanation producing prompts (particularly rationale first / CoT style) affect detection/localization/patching performance. Only qualitative trends stated in the excerpts are included.

Ref	Task	Compared prompt/pipeline variants	Reported effect of CoT / rationale first prompting on task performance
[1]	Detection	Standard vs Step by Step (CoT-like) vs Definition based, various prompting techniques	CoT like prompting exists but is "brittle" and sensitive to small code changes; reasoning often incorrect even when vulnerability is correctly flagged. No claim that CoT reliably improves accuracy.
[2]	Detection	Basic, In Context, CoT SA, Contrastive Pairs	CoT and contrastive prompts are explored to maximize performance; Balanced Accuracy remains modest (~0.50–0.63) and models fail to distinguish buggy vs fixed in many cases; no clear overall benefit from CoT reported in the excerpt.
[3]	Detection (real CVEs)	Standard, P CoT, P Decomp, P P&S; ZS vs FS	P Decomp performs consistently; P CoT and P P&S improve results overall; GPT 4 best with Plan and Solve; reasoning often fails to generalize to unfamiliar vulns. Overall: structured reasoning prompts can improve detection accuracy in this setting.
[4]	Detection		Focus is on contrastive reasoning quality; the excerpt does not explicitly state whether CoT helps or

		Context rich CoT detection prompt vs other settings (details in appendix)	harms detection accuracy versus non CoT prompts.
[5]	Detection	Vulnstruct vs baselines; structured reasoning vs non specification baselines	Reports large F1/recall gains under “strict criteria requiring both correct predictions and valid reasoning”; suggests structured reasoning prompts improve both detection and reasoning quality.
[6]	Detection + explanation	Instruction tuned LLMVulExp vs base models; CoT based key code extraction vs simpler prompts	Experiments show improved detection and explanation outputs, but the excerpt does not separate the effect of CoT style prompting from instruction tuning and other design choices.
[7]	Detection, explanation, patching	Multi agent (separate detection, explanation, patch agents) vs single agent baseline	Multi agent with explicit explanation stage significantly improves performance across sub tasks versus a monolithic agent; reports 93.3% accuracy/F1 and high explanation and patch validity.
[8]	Repair	VRpilot pipeline with rationale first CoT + patch validation feedback vs ablations	+14% correct patches for C and +7.6% for Java over baselines; ablation shows both CoT reasoning stage and validation feedback are critical. Strong evidence that rationale first prompting plus feedback improves patching.
[9]	Patching	APPATCH CoT exemplars vs other prompting/repair baselines and non LLM tools	Reports large gains in F1/recall and successful patches for real world vulnerabilities; preliminary experiment suggests reasoning helps patch generation, but effect of CoT vs non CoT not isolated in detail in the excerpt.
[10]	Identification, discovery, patching	VSP CoT exemplars vs five baselines, including zero shot VSP and a removed naive CoT baseline	VSP achieves substantial F1/accuracy improvements across tasks; zero shot VSP worse than few shot; naive line by line CoT did not work well. CoT is beneficial when carefully structured around “vulnerability semantics,” not when naively applied.
[11]	Detection	CoT vs few shot vs in context; multiple context abstraction levels	Primitive API abstraction improves accuracy and reduces false positives; different vulns prefer different abstraction levels. CoT is included in the mix but, per the excerpt, the main effect comes from context abstraction, not necessarily CoT.
[12]	Detection + reasoning	Various prompt schemes (including CoT, Post CoT), plus tool/knowledge variants	Provides recommendations on prompt schemes; details in the excerpt do not specify whether CoT consistently improves accuracy, but the framework is explicitly used to understand how prompt schemes affect both reasoning and prediction.
[13]	Localization	Direct, few shot CoT, Mixture of Agents, rStar	CoT is used as a baseline; rStar and MoA are designed to improve reasoning/localization; excerpt emphasizes limits of CoT on long contexts. CoT does not appear to be the best performing regime.
[14]	Localization	Same as [13] plus line by line checking	Shows that simple line by line checking with attention improves localization and mitigates reasoning degradation; CoT baseline is included but not superior.
[15]	Detection	Dataflow prompt, CoT (“let’s think step by step”), and “dataflow + self-reflection”	CoT does not improve over the dataflow prompt; few shot examples not particularly helpful. Self-reflection adds complexity without clear net gain in the excerpt.

[16]	Detection	Basic, code specific info, and CoT prompts; also position of info and temperature	Combined prompts with code specific information and CoT are explored; results show prompt content/structure matter, but the excerpt does not isolate the effect of CoT alone.
------	-----------	---	---

Key patterns:

- For vulnerability detection:
 - Carefully structured reasoning prompts (e.g., decomposition or plan and solve in [3], specification guided reasoning in [5]) can improve performance.
 - Generic “let’s think step by step” CoT often fails to help or is brittle ([1,2,15]) and sometimes underperforms structured dataflow prompts [15].
- For repair/patching:
 - Rationale first or multi step reasoning that explicitly analyzes root cause and strategy before patching yields clear gains in multiple systems: VRpilot [8], APPATCH [9], VSP [10], and the multi agent VulnPatch [7].
- For localization:
 - CoT style reasoning is at best a baseline and can degrade with long contexts; attention based or multi trajectory methods (MoA, rStar, line by line checking) are more promising [13,14].

Synthesis: What is Unique to Rationale First Prompting in This Literature

Where Explain Before Answer is Most Central

- Patch/repair pipelines:
 - VRpilot [8], APPATCH [9], VSP [10], and VulnPatch [7] all embed explicit reasoning stages—root cause analysis, fixing strategy, or explanation agents—before patch generation.
 - Among these, [8] provides the clearest causal evidence via ablation that a rationale first stage, combined with external feedback, materially improves patch correctness.
- Vulnerability detection with contrastive or specification guided reasoning:
 - CORRECT [4] and Vullnstruct [5] structure prompts around root cause evidence or security specifications; rationales are required to be consistent with those structures.
 - LLMVulExp [6] goes further by defining actionability as an explicit quality dimension for explanations.
- Benchmarks focused on “vulnerability reasoning”:
 - LLM4Vuln [12] and CORRECT [4] explicitly frame their contribution as dissecting vulnerability reasoning apart from external aids, rather than merely measuring classification accuracy.

Where CoT/Rationale First is a Controlled Variable vs Background Detail

- In [1,2,3,11,15], and [16], CoT or rationale like prompting is primarily a prompt engineering factor in detection experiments. These works:
 - Compare multiple prompt strategies, including CoT and non CoT.
 - Show that:
 - CoT is not reliably beneficial for detection (especially compared to domain structured prompts) [1,2,15].
 - Structured decomposition or domain aligned abstractions can be more important than just “thinking step by step” [3,5,11].
- In [13] and [14], reasoning oriented prompting (CoT, MoA, rStar) is part of the decoder design space for localization, but the key technical novelty lies in attention based line by line checking.

Where Explanation Quality is First Class vs Secondary

- First class:
 - [1] (faithful reasoning dimension and robustness), [2] (error taxonomy), [3] (joint reasoning+accuracy score), [4] (LLM as - judge against CVE/CWE/diff), [6] (accuracy/clarity/actionability), [7] (sound reasoning %, albeit coarsely), [12] (vulnerability reasoning capability).
- Secondary or implicit:
 - [5] (valid reasoning as a success condition, manual failure analysis), [8,9,10,11,13,14,15,16] (reasoning discussed qualitatively or indirectly via task performance, but not independently scored).

For your specific goal—understanding how requiring explanations before the main output affects both explanation quality and task performance in vulnerability detection and code generation—the most directly informative papers are:

- Vulnerability detection & explanation quality:
 - [1,2,3,4,5,6,12,15].
- Vulnerability repair/patching with explicit rationale first protocols:
 - [7,8,9,10] (plus the detection side works above when they have EBA like prompts).
- Works that explicitly tie explanation quality to task performance under different prompting schemes:
 - [3] (joint score for accuracy and reasoning quality),
 - [4] (contrastive evaluation of rationales on vulnerable/patched pairs),
 - [5] (strict prediction+reasoning correctness),
 - [6] (actionability vs detection),
 - [7] and [8] (performance differences with vs without explicit reasoning stages).

Timeline

Chronological Phases in Rationale First / CoT Research for Vulnerability Tasks

1. Early LLM Vulnerability Evaluations with CoT (H2023–early 2024)

The first wave of security oriented LLM work brought CoT / “explain then judge” into vulnerability detection, but treated rationales mostly as a diagnostic by product rather than a systematically evaluated object.

Key milestones:

- **SecLLMHolmes – security & privacy framing of LLM reasoning limits**
 - Ullah et al. [1] (SP 2024; 2023 preprint) provide one of the earliest comprehensive evaluations of LLMs for vulnerability detection, explicitly comparing prompting styles including:
 - Standard (direct answer),
 - Step by Step reasoning (security expert style CoT),
 - Definition based.
 - They measure multiple axes (accuracy, robustness to code edits, “faithful reasoning”, etc.) and show:
 - High false positive rates and brittleness of CoT reasoning,
 - Many cases where a vulnerability is correctly flagged but the accompanying rationale is wrong or unfaithful.
 - However, they do not enforce a strict “rationale block then answer block” protocol nor quantitatively operationalize usefulness/faithfulness of explanations; explanation assessment is largely qualitative/robustness style [1].
- **Prompt engineering studies for vulnerability detection**
 - Khare et al. [15] (ICST 2025; 2023 preprint) systematically compare four prompting strategies (including a dataflow style prompt and a step by step CoT prompt) across 16 LLMs and five datasets.
 - They explicitly require that CoT prompts “think step by step” before final verdicts.
 - They show CoT does not consistently outperform strong task specific prompts, and few shot examples add limited value [15].
 - Reasoning chains are qualitatively inspected but not quantitatively scored for correctness or faithfulness.
 - These early studies set up the core observation that:
 - “Explain before answer” can change detection outcomes,
 - But explanation quality is often poor and is not yet rigorously measured.

Overall, this phase establishes that:

- CoT/rationale style prompting is now standard in evaluating LLM vulnerability detection,
- Early work focuses on task metrics and simple robustness/qualitative reasoning checks rather than formal explanation quality metrics.

2. Systematic Prompt & Context Studies for Detection (H2024)

The next cluster of work broadens prompt design and code context, still mostly focusing on detection performance, with rationales present but not fully quantified.

- **Prompt and context engineering for detection**

- Liu et al. [16] explore basic prompts, prompts with code specific information (similar code, data flow graphs), and CoT prompts for vulnerability detection.
 - They also vary the position of code specific information and temperature.
 - CoT is introduced “to inspire GPT’s ability for gradually detecting vulnerability” [16].
 - Explanations are used to structure reasoning but not evaluated as such.
- Yang et al. [11] build a context enhanced detection approach using control/data flow–derived context and compare:
 - CoT prompts that first ask for detailed code analysis (summaries, pointer behaviors), then vulnerability judgments,
 - Other prompting strategies (few shot, in context learning).
 - They show that context abstraction level and CoT prompts materially affect detection performance, especially across vulnerability types [11].
 - Again, explanations are not scored for usefulness/faithfulness.
- **Decoupling and benchmarking “vulnerability reasoning”**
 - Sun et al. [12] introduce LLM4Vuln and the UniVul benchmark to decouple vulnerability reasoning from external aids across Solidity, Java, C/C++.
 - They always elicit a structured output of (a) label + (b) rationale, and explore multiple prompt schemes:
 - Including variants where CoT is requested before or after the decision (“Post CoT”) [12].
 - They analyze how prompt schemes, external tools, and knowledge affect performance but do not report explicit explanation quality metrics (they mainly use GPT 4 to structure outputs and evaluate task level reasoning capability).

Conceptually, this phase refines the understanding that:

- Prompt style (CoT, dataflow like prompts, context rich prompts) and contextual information significantly impact vulnerability detection performance [11,12,15,16].
- Rationale first / CoT prompting is routinely used, but explanations are still primarily treated as scaffolding for the model, not as rigorously evaluated artifacts.

3. Rationale First Pipelines for Repair and Patching (H2024–2025)

A distinct strand focuses on vulnerability repair/patching, where rationale first prompting is much more explicit and methodologically central.

- **VRpilot – explicit two stage rationale first pipeline**
 - Kulsum et al. [8] (VRpilot) is one of the clearest “explain before patch” systems:
 - Stage 1: a CoT prompt elicits a structured reasoning trace (root cause, impact, possible fixes).
 - Stage 2: a second prompt extracts or uses this reasoning to generate a patch.
 - The pipeline is iteratively refined using external patch validation feedback (compilers, sanitizers, tests).
 - They show:
 - +14% correct patches for C and +7.6% for Java over baselines,
 - Ablation studies where removing the CoT stage or patch validation degrades performance, indicating that rationale - first prompting has a causal effect on patch quality [8].
 - Crucially, they do not score or analyze the explanations themselves beyond “improves patching”; explanation quality is implicit.
- **APPATCH / LLMPATCH – CoT structured exemplars for patching**
 - Nong et al. [9] (USENIX Security 2024) develop APPATCH:
 - Few shot exemplars are automatically constructed to include a 3 step rationale:
 1. root cause analysis, 2) fixing strategy, 3) patch generation [9].
 - At inference, the model is guided to follow this rationale first pattern before producing patches.
 - They report large F1/recall gains on real world vulnerabilities and Big Vul, as well as successful zero day patches [9].
 - The reasoning is described as “helpful to guide patch generation” but not quantitatively evaluated for correctness/faithfulness.
- **VSP – vulnerability semantics guided CoT for detection and patching**
 - Nong et al. [10] (the same group) generalize to a broader Vulnerability Semantics guided Prompting (VSP) strategy:
 - They design few shot CoT exemplars centered on “vulnerability semantics” (root cause behaviors, key statements), and use them across:
 - Vulnerability identification (specific type),
 - Vulnerability discovery (any type),

- Vulnerability patching [10].
- For patching, they use a two step rationale first prompt: “Root cause analysis” then “Patching strategy” [10].
- They report consistent F1/accuracy improvements over multiple baselines.
- They also document negative results for an overly fine grained line by line CoT style, which failed, suggesting a nuanced relationship between rationale granularity and effectiveness.

Collectively, these repair oriented works mark a major methodological shift:

- From “CoT vs direct answer” evaluations to integrated, multi stage pipelines where reasoning is structurally required before code generation.
- They provide stronger evidence that rationale first prompting can causally improve task performance (patch correctness), even though explanation quality metrics remain underdeveloped.

4. Increasing Focus on Explanation Aware Detection Frameworks (H2024–2025)

Recent frameworks move closer to jointly considering detection performance and structured evaluation of rationales.

- **CORRECT – contrastive rationale assessment on paired vulnerable/patched code**
 - Li et al. [4] present CORRECT, an evaluation framework and dataset of 2,000 vulnerable–patched pairs covering 99 CWEs.
 - Prompt I: context rich vulnerability assessment with zero shot CoT, eliciting both judgment and rationale.
 - Prompt II: rationale assessment – they feed model rationales plus ground truth evidence (CVE, CWE, commit message, diff) to an LLM as judge to evaluate alignment [4].
 - Rationales must include the root cause for vulnerable versions and avoid false alarms for patched versions [4].
 - This is one of the first security works to:
 - Explicitly state rationale requirements (root cause completeness),
 - Use a contrastive vulnerable vs patched setting to evaluate explanation correctness.
 - Although the evaluation uses LLM as judge rather than human labels, it represents a concrete operationalization of explanation “correctness/usefulness” with respect to ground truth evidence.
- **Real VulLLM – in the wild CVE evaluation with reasoning quality scores**
 - Safdar et al. [3] propose Real VulLLM for real world CVEs:
 - Compare multiple reasoning oriented prompts (Standard, CoT, Decomposition, Plan and Solve) in both zero shot and few shot settings.
 - Introduce a combined scoring metric that jointly accounts for:
 - Classification accuracy (vulnerable vs secure),
 - “Reasoning quality” of the explanations [3].
 - Empirical findings:
 - CoT and Plan and Solve generally improve performance, with Plan and Solve + GPT 4 doing best,
 - Reasoning is often incomplete or incorrect even when classification is right, especially for unfamiliar vulnerability patterns [3].
 - While details of the reasoning quality metric are not fully described in the excerpt, this represents a move toward explicit, quantitative scoring of explanations in the vulnerability context.
- **LLMVulExp – explanation centric detection and explanation metrics**
 - Mao et al. [6] go one step further by:
 - Instruction tuning LLMs (LoRA) to jointly detect vulnerabilities and generate explanations (cause, location, repair suggestions).
 - Designing structured prompts with CoT based “key code extraction” to steer reasoning [6].
 - Proposing an evaluation protocol for explanations along three dimensions:
 - Accuracy, clarity, and actionability,
 - Using automated LLM based scoring with expert verification [6].
 - This is one of the clearest examples of explicit explanation quality metrics (albeit LLM judged) in vulnerability detection, aligned with actionable security criteria.
- **Specification guided reasoning**
 - Zhu et al. [5] introduce VulInstruct, which uses “security specifications” as structured rationales:
 - Extracts general and domain specific security specifications from patches/CVEs,
 - Guides LLMs to infer and apply these specifications during detection,

- Evaluates under “strict criteria requiring both correct predictions and valid reasoning” [5].
- They perform manual failure analyses of reasoning (e.g., zero reasoning, miscategorization, over generalization), but the notion of “valid reasoning” is only loosely defined in the excerpt.

This phase marks a qualitative step toward:

- Treating rationales as first class objects with operationalized evaluation (even if via LLM as judge),
- Designing prompts and datasets that explicitly require root cause and localization information in explanations.

5. Rationale First in Localization and Multi Agent Security Systems (H2024–2025)

Parallel to detection and patching, rationale first ideas are explored in localization and multi agent systems.

- **LOVA – reasoning aware localization with attention and CoT**
 - Li et al. develop LOVA in two closely related papers:
 - “Attention Is All You Need for LLM-based Code Vulnerability Localization” [13],
 - “If LLMs Would Just Look: Simple Line-by-line Checking Improves Vulnerability Localization” [14].
 - They:
 - Use decoder self attention to identify vulnerable lines, and
 - Compare four prompting/decoding regimes:
 - Vanilla (direct),
 - Few shot CoT where the model “first outputs the reasoning process before providing the final answer” (explicit rationale first),
 - Mixture of Agents (MoA),
 - rStar (multiple reasoning trajectories, select consistent one) [13,14].
 - Findings:
 - CoT helps in some settings but degrades on long contexts (supporting “overthinking” and degradation concerns),
 - Alternative mechanisms (attention, line by line checking) can outperform CoT for localization [13,14].
 - Explanation quality per se is not scored, but these works position CoT rationales as one component in a broader toolbox for vulnerability localization.
- **Multi agent rationale pipelines for smart contracts**
 - Sakib and Khan [7] propose VulnPatch, a multi agent framework for smart contracts, with agents specializing in:
 - Detection,
 - Explaining causes,
 - Generating patched code [7].
 - They report:
 - 93.3% accuracy and F1,
 - Explanations and patches showing “90% sound reasoning” and “84% patch validity” [7].
 - The notion of “sound reasoning” suggests some explanation quality assessment, but the excerpt does not detail the evaluation protocol (human vs automated, criteria).

These works extend rationale first concepts:

- Beyond binary detection/patching to line level localization and multi agent workflows,
- Highlighting trade offs between CoT and alternative mechanisms like attention based heuristics.

Emerging Trends and Shifts in Focus

From “Does CoT Help?” to “How Should Reasoning Be Structured?”

- Early works [1,15,16] treat CoT as a monolithic “think step by step” toggle and simply ask whether it improves accuracy.
- Later works introduce more nuanced reasoning structures:
 - Decomposition and Plan and Solve prompts [3],
 - Vulnerability semantics guided CoT [10],
 - Root cause ‘strategy’ patch multi step rationales [9],

- Specification guided reasoning [5],
- Line by line checking vs global CoT [14].
- Negative results (e.g., failure of line by line CoT in VSP [10], CoT not beating dataflow prompts [15], CoT degrading on long contexts [13,14]) shift the conversation from “more reasoning tokens” to “better targeted reasoning structures.”

From Qualitative Reasoning Inspection to Quantified Explanation Metrics

- Earliest security works only qualitatively note that explanations are often wrong or inconsistent [1,15].
- Intermediate frameworks like LLM4Vuln [12] focus on structuring outputs and differentiating prompt schemes, but still primarily measure task accuracy.
- Recent works start to explicitly quantify explanation quality:
 - LLMVulExp's accuracy/clarity/actionability metrics with LLM as judge [6],
 - CORRECT's contrastive rationale assessment against CVE/CWE/diff evidence [4],
 - Real VulLLM's combined score that explicitly includes reasoning quality components [3],
 - VulnPatch's reported “90% sound reasoning” for explanations [7].
- This trend aligns with the broader concern in your background: operationalizing “usefulness” and “faithfulness” beyond task metrics.

From Single Shot Answers to Multi Stage and Multi Agent Pipelines

- Early work: single shot classification with optional CoT [1,15,16].
- Repair focused systems like VRpilot and APPATCH [8,9] introduce:
 - Two stage rationale first prompts (reason ' patch),
 - Iterative refinement using external feedback (tests, sanitizers).
- Localization work [13,14] and multi agent smart contract frameworks [7] introduce:
 - Multiple interacting agents/decoding processes,
 - Different roles (detector vs explainer vs patcher),
 - Voting/selection over multiple reasoning trajectories.
- This multi stage/multi agent shift opens more opportunities to:
 - Treat rationales as explicit intermediate artifacts,
 - Manipulate them (e.g., self refine, cross check) before final code/decisions.

From Benchmarking “Lab” Vulnerabilities to In the Wild and Context Rich Evaluations

- Early datasets: synthetic or benchmark oriented vulnerability sets (e.g., Big Vul, PrimeVul, D2A) [2,5,8,9,10,15,16].
- Newer frameworks deliberately move to:
 - Real world CVEs (Real VulLLM [3]),
 - Context rich shared graphs of real vulnerable projects (CORRECT [4]),
 - Multi language, multi ecosystem benchmarks (UniVul in LLM4Vuln [12]).
- This makes explanation evaluation more realistic:
 - Rationales must integrate contextual evidence (CVE narratives, diffs) [4],
 - Prompt designs must cope with long, non local code contexts [11,13,14].

Research Clusters, Key Contributors, and Their Trajectories

Several research groups have sustained, coherent threads that shape the field's development.

Haipeng Cai and collaborators: VSP and APPATCH [9,10]

- Contributions:
 - Introduced vulnerability semantics guided CoT prompting (VSP) across detection and patching tasks [10].
 - Built APPATCH/LLMPATCH with automated CoT exemplars and three step rationales guiding patch generation [9].
- Impact:
 - Pioneered using structured, vulnerability specific reasoning exemplars rather than generic “think step by step.”

- Demonstrated tangible patching gains, influencing later repair pipelines and setting a de facto template for rationale first patching.
- Future direction:
 - Likely to extend toward more explicit evaluation of rationale faithfulness and integration of external analysis tools, given their focus on “vulnerability semantics.”

Yue Li, Sheng Zhong, and collaborators: Localization and CORRECT [4,13,14]

- Contributions:
 - LOVA localization method with:
 - Attention based line level reasoning [13,14],
 - Systematic comparison of CoT, MoA, and rStar decoding schemes.
 - CORRECT framework and dataset for context rich vulnerability detection and contrastive rationale assessment [4].
- Impact:
 - Provide both:
 - A methodological critique of CoT for long context localization (emphasizing attention based alternatives),
 - One of the first structured, contrastive explanation evaluation pipelines (CORRECT).
- Future direction:
 - Their combined focus on localization, context rich evaluation, and rationale assessment suggests further work on aligning rationales with precise program slices and static analysis abstractions.

Yang Liu and collaborators: LLM4Vuln [12]

- Contributions:
 - UniVul benchmark and LLM4Vuln evaluation framework to decouple vulnerability reasoning from external aids.
 - Systematic comparison of prompt schemes, including pre/post decision CoT variants [12].
- Impact:
 - Shifted attention from “which model is best” to “how prompt schemes/tools interact with reasoning ability,” a prerequisite to serious methodology for rationale first prompting.
- Future direction:
 - Natural fit to connect with emerging explanation quality metrics (as in CORRECT, LLMVulExp), and to study faithfulness via perturbation/contrastive tests.

VRpilot / repair oriented group: d’Amorim et al. [8]

- Contributions:
 - VRpilot – explicit two stage rationale first repair pipeline with strong ablation evidence for the value of CoT reasoning and validation feedback [8].
- Impact:
 - Provides one of the clearest empirical demonstrations that explanation first prompting, when combined with external feedback, improves security critical code generation.
- Future direction:
 - A natural next step would be to measure how explanation quality (e.g., correctly identifying root causes) correlates with patch success and to design evaluation protocols accordingly.

Explanation centric frameworks: Mao et al. (LLMVulExp) [6], Safdar et al. (Real VulLLM) [3], Zhu et al. (VulInstruct) [5]

- Contributions:
 - LLMVulExp: instruction tuning for explanations plus accuracy/clarity/actionability metrics [6].
 - Real VulLLM: real world CVE evaluation with joint accuracy + reasoning quality scoring [3].
 - VulInstruct: specification guided reasoning with “valid reasoning” criteria and failure analyses [5].
- Impact:
 - Collectively, they start to define what “good explanations” look like in vulnerability tasks and how to score them, which is central to your interest in usefulness and faithfulness.

- Future direction:
 - Likely to inspire datasets with human annotated rationales and more fine grained metrics (e.g., localization precision, source–sink coverage, assumption tracking).

Implications for the Future of Rationale First Research in Vulnerability Tasks

Synthesizing the timeline and patterns:

1. Increasing methodological maturity:

- The field is moving from simple “CoT vs direct answer” comparisons [1,15,16] toward:
 - Structured multi stage rationale first pipelines [8,9,10],
 - Explanation quality metrics aligned with security tasks [3,4,5,6],
 - Contrastive and context rich evaluation setups [3,4,12].
- However, robust faithfulness tests (perturbation/counterfactuals, behavioral grounding) are still largely missing, even in the latest work.

1. Growing recognition of trade offs:

- Evidence now shows:
 - CoT can be brittle, overly verbose, and sometimes harmful in long context settings [1,13,14].
 - Task specific reasoning structures (vulnerability semantics, security specifications, line by line checking) can outperform generic “think step by step” prompts [5,10,14].
- Future designs are likely to:
 - Combine CoT with program analysis (as in [11]) and attention based signals [13,14],
 - Use structured rationales (slots for source, sink, sanitization, root cause) to reduce hallucination and improve evaluability.

1. Convergence of detection, localization, and repair:

- Early work treated detection, localization, and patching separately, but:
 - VSP/APPATCH and VRpilot integrate detection like reasoning with patch synthesis [8,9,10],
 - LLMVulExp and VulnInstruct unify detection with explanation and repair suggestions [5,6],
 - LOVA bridges localization and reasoning [13,14].
- This suggests future “end to end” frameworks where:
 - Rationale first prompting is used consistently across detect’localize’explain’patch,
 - Explanation quality is evaluated at each stage (e.g., localization accuracy, root cause correctness, alignment between rationale and patch).

1. Opportunity for explicit faithfulness research:

- While some papers talk about “faithful reasoning” [1] or “valid reasoning” [5], systematic, behaviorally grounded faithfulness tests (as in your background) are still largely unaddressed.
- Given the emerging explanation evaluation infrastructure (CORRECT, LLMVulExp, Real VulLLM) [3,4,6], a natural next wave of work could:
 - Perform counterfactual perturbation tests on code elements highlighted in rationales,
 - Compare multiple prompts (answer first vs rationale first vs hidden CoT) within the same model, controlling for tokens, to isolate causal effects on both performance and explanation quality.

In sum, over roughly two years the field has evolved from exploratory CoT vs direct prompts for vulnerability detection to sophisticated, rationale first, multi stage pipelines and frameworks that begin to explicitly measure explanation quality. The next phase will likely focus on stronger operationalizations of usefulness and faithfulness, tighter coupling of rationales with program semantics, and rigorous causal studies of how explain before answer prompting shapes both LLM behavior and human in the loop security workflows.

Foundational Work

Which papers form the foundational references on this topic?

The below table shows the resources that are most often cited by the relevant papers on this topic. This is measured by the **reference rate**, which is the fraction of relevant papers that cite a resource. Use this table to determine the most important core papers to be familiar with if you want to deeply understand this topic. Some of these core papers may not be directly relevant to the topic, but provide important context.

Ref.	Reference Rate	Title	Cited By These Relevant Papers
[12]	0.59	LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning	[3, 4, 5, 6, 7]
[27]	0.49	Vulnerability Detection with Code Language Models: How Far are We?	[4, 5, 6, 11, 12]
[141]	0.46	VulnLLMEval: A Framework for Evaluating Large Language Models in Software Vulnerability Detection and Patching	[3, 4]
[34]	0.34	Boosting Vulnerability Detection of LLMs via Curriculum Preference Optimization with Synthetic Reasoning Data	[5]
[108]	0.34	A Survey on Code Generation with LLM-based Agents	[5]
[10]	0.34	Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities	[5, 6, 9, 13]
[36]	0.27	Chain of Thought Prompting Elicits Reasoning in Large Language Models	[6, 8, 10, 12]
[79]	0.25	LineVul: A Transformer-based Line-Level Vulnerability Prediction	[3, 6, 10]
[15]	0.24	Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities	[3, 4, 12]
[63]	0.24	How Far Have We Gone in Vulnerability Detection Using Large Language Models	[3, 6, 12]
[29]	0.23	LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights	[4]
[68]	0.23	Do advanced language models eliminate the need for prompt engineering in software engineering?	[4]
[14]	0.23	If LLMs Would Just Look: Simple Line-by-line Checking Improves Vulnerability Localization	[5]
[32]	0.23	R2Vul: Learning to Reason about Software Vulnerabilities with Reinforcement Learning and Structured Reasoning Distillation	[5]
[83]	0.23	aiXcoder-7B-v2: Training LLMs to Fully Utilize the Long Context in Repository-level Code Completion	[5]
[142]	0.23	Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories	[5]
[96]	0.23	JudgeBench: A Benchmark for Evaluating LLM-based Judges	[4]
[17]	0.23	Beyond ChatGPT: Enhancing Software Quality Assurance Tasks with Diverse LLMs and Validation Techniques	[5]
[25]	0.22	Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study	[5, 11]
[86]	0.21	Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection	[3, 4]

Adjacent Work

Which papers cite the same foundational papers as relevant papers?

Use this table to discover related papers on adjacent topics, to gain a broader understanding of the field and help generate ideas for useful new research directions.

Ref.	Adjacency score	Title	References These Foundational Papers
[32]	1.92	R2Vul: Learning to Reason about Software Vulnerabilities with Reinforcement Learning and Structured Reasoning Distillation	[10, 12, 18, 25, 27, 116]
[148]	1.65	When LLMs meet cybersecurity: a systematic literature review	[8, 12, 15, 22, 25]

[81]	1.61	Generative transformations and patterns in LLM-native approaches for software verification and falsification	[10, 12, 15, 22]
[149]	1.56	From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security	[8, 10, 22, 25, 27]
[25]	1.52	Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study	[10, 12, 15, 18, 36]
[29]	1.38	LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights	[8, 15, 27, 46, 53, 73, 123]
[27]	1.26	Vulnerability Detection with Code Language Models: How Far are We?	[12, 15, 36, 73, 119]
[150]	1.16	Revisiting Pre-trained Language Models for Vulnerability Detection	[12, 15, 27, 123]
[26]	1.08	Detecting Code Vulnerabilities using LLMs	[15, 18, 46, 53, 73, 116, 119]
[151]	1.05	Safe4U: Identifying Unsound Safe Encapsulations of Unsafe Calls in Rust using LLMs	[12, 15, 27]
[152]	1.05	Large Language Models for Security Operations Centers: A Comprehensive Survey	[22, 25, 27, 29]
[77]	1.05	A Mixture of Linear Corrections Generates Secure Code	[12, 15, 27]
[15]	1.02	Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities	[1, 27, 36, 73, 109, 119, 123]
[153]	1.01	Vul-R2: A Reasoning LLM for Automated Vulnerability Repair	[8, 12, 27]
[154]	1.01	Frontier AI's Impact on the Cybersecurity Landscape	[8, 12, 27]
[12]	1.01	LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning	[15, 27, 36, 73, 116]
[101]	0.99	PVDetector: Pretrained Vulnerability Detection on Vulnerability-enriched Code Semantic Graph	[10, 12, 123]
[37]	0.96	Can LLM Prompting Serve as a Proxy for Static Analysis in Vulnerability Detection	[1, 15, 27, 46, 119]
[155]	0.94	jWith a Little Help from My (LLM) Friends: Enhancing Static Analysis with LLMs to Detect Software Vulnerabilities	[10, 15, 27]
[156]	0.91	Can LLMs handle WebShell detection? Overcoming Detection Challenges with Behavioral Function-Aware Framework	[10, 12]

References

- [1] [LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities \(Yet?\): A Comprehensive Evaluation, Framework, and Benchmarks](#)
Saad Ullah, ..., and Gianluca Stringhini. 2024 IEEE Symposium on Security and Privacy (SP), 2023. 113 citations.
100% Topic Match
Demonstrates a comprehensive empirical evaluation of LLMs on security vulnerability identification.
Builds SecLLMHolmes and tests 228 C/Python scenarios across 8 CWE classes, 8 LLMs, and multiple prompt styles (Standard, Step by Step CoT, Definition based).
Relevant because it compares prompting styles and measures reasoning faithfulness/robustness, but it's unclear if it enforces strict explain before answer protocols or uses explicit usefulness/localization metrics.
- [2] [To Err is Machine: Vulnerability Detection Challenges LLM Reasoning](#)
Benjamin Steenhoek, ..., and Wei Le. ArXiv, 2024. 18 citations.
100% Topic Match
Demonstrates a systematic evaluation of LLMs on vulnerability detection and their generated explanations.
Tests 11 code-capable LLMs across five prompt templates (including CoT-style and contrastive prompts), multiple datasets, and analyzes 287 model explanations and failure modes.
Relevant because it compares prompting styles and evaluates explanation content (error taxonomy, localization, root-cause reporting), but does not clearly enforce a strict explain-before-answer protocol nor perform explicit faithfulness tests (e.g., perturbation/counterfactuals).
- [3] [Real-VulLLM: An LLM Based Assessment Framework in the Wild](#)
Rijha Safdar, ..., and Wajahat Hussain. ArXiv, 2025. 0 citations.
100% Topic Match
Presents an empirical framework comparing explain-before-answer style prompts for vulnerability detection.
Evaluates multiple prompting strategies (no-reasoning, P CoT, P Decomp, P P&S) in zero-/few-shot across several LLMs on real CVEs with an NVD vector datastore.
Reports a joint accuracy+reasoning-quality score, finds reasoning often incomplete/incorrect despite correct classification, P Decomp stable, P P&S/P CoT improve overall; includes per CVE variability and notes limits for real world deployment.
- [4] [Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask](#)
Yue Li, ..., and Sheng Zhong. ArXiv, 2025. 9 citations.
100% Topic Match
Proposes a context-rich evaluation framework (CORRECT) for LLM-based vulnerability detection that elicits binary judgments and natural-language rationales.
Builds a 2,000 vulnerable-patched program-pair dataset (99 CWEs), extracts execution context via code-property-graph slicing, then prompts 13 LLMs to give HASVUL/NOVUL plus zero-shot chain-of-thought rationales; uses an LLM-as-judge to validate rationale alignment with ground-truth evidence.
Relevant: explicitly enforces explain-before-or-with-answer rationales, performs contrastive vulnerablepatched sensitivity tests, and evaluates rationale quality (alignment/faithfulness) rather than only task accuracy—good match to your criteria.
- [5] [Specification-Guided Vulnerability Detection with Large Language Models](#)
Hao Zhu, ..., and Ge Li. ArXiv, 2025. 0 citations.
100% Topic Match
Proposes a specification-guided LLM method (VulInstruct) for vulnerability detection.
Builds a KB of general and domain-specific security specifications, retrieves past cases, and prompts structured chain-of-thought reasoning to align/specify vulnerabilities.
Reports strict evaluation requiring "correct predictions and valid reasoning" with large F1/recall gains, but does not clearly enforce explain-before-answer prompting nor provide explicit operationalized usefulness/faithfulness metrics (only manual failure analyses).
- [6] [Towards Explainable Vulnerability Detection With Large Language Models](#)
Qiheng Mao, ..., and Jianling Sun. IEEE Transactions on Software Engineering, 2024. 2 citations.
99% Topic Match
Proposes LLMVulExp: an instruction-tuned LLM framework for joint vulnerability detection and explanation.
Uses prompt-based annotation, LoRA instruction fine-tuning, CoT-based key-code extraction, and structured prompts to produce detection + natural-language explanations (cause, location, repair).
Evaluation measures explanation accuracy/clarity/actionability via automated LLM scoring with expert verification, but paper does not report an explicit explain-before-answer (rationale-first) condition or behavioral/faithfulness causal tests.
- [7] [VulnPatch: Multi-Agent Automated Smart Contract Vulnerability Detection, Explanation, and Mitigation Framework](#)
Md. Nazmus Sakib and Latifur Khan. 2025 7th International Conference on Blockchain Computing and Applications (BCCA), 2025. 0 citations.
93% Topic Match
Proposes a multi-agent LLM pipeline for detect-explain-patch of smart-contract vulnerabilities.
Achieves this by fine-tuning separate LLM agents for detection, cause explanation, and patch generation, then aggregating reports.
Reports empirical gains versus single-agent baselines (93.3% accuracy/F1; 84.44% report coverage; explanations 90% sound; patches 84% valid), but does not state whether explanations are produced strictly before final outputs, nor whether explanation faithfulness/usefulness is operationalized with localization/behavioral tests.
- [8] [A Case Study of LLM for Automated Vulnerability Repair: Assessing Impact of Reasoning and Patch Validation Feedback](#)
Ummay Kulsum, ..., and Marcelo d'Amorim. Proceedings of the 1st ACM International Conference on AI-Powered Software, 2024. 34 citations.
88% Topic Match
Demonstrates a rationale-first CoT pipeline for automated vulnerability repair.
Implements a two-stage prompt (reasoning CoT then patch extraction) plus iterative external patch-validation feedback to generate fixes.
Evaluated on public C/Java datasets (VRpilot) with +14%/+7.6% correct patches vs baselines; ablation shows both CoT and validation matter — but no explicit operationalization or evaluation of explanation quality (usefulness or faithfulness) or line-level localization of rationales.
- [9] [APPATCH: Automated Adaptive Prompting Large Language Models for Real-World Software Vulnerability Patching](#)
Yu Nong, ..., and Haipeng Cai. Unknown journal, 2024. 12 citations.
68% Topic Match
Demonstrates an adaptive, rationale-first LLM prompting pipeline for automated vulnerability patching.
Uses CoT-style three-step exemplars (root-cause analysis 'fixing strategy' patch generation) auto-generated from templates to prompt LLMs and produce patches; evaluated on 97 zero-day + 20 known vulnerabilities with large F1/recall gains.
Relevant: it enforces explain-before-answer prompting and shows improved patch performance, but does not provide explicit quantitative evaluation of explanation quality (usefulness or faithfulness) or behavioral/faithfulness tests—explanations are reported as "helpful" but not systematically scored.
- [10] [Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities](#)
Yu Nong, ..., and Haipeng Cai. ArXiv, 2024. 49 citations.
44% Topic Match
Proposes a CoT-inspired prompting method (VSP) for vulnerability identification, discovery, and patching.
Implements few-shot exemplars mapping "vulnerability semantics" into stepwise reasoning prompts; compares VSP to five baselines across three LLMs and two datasets, reporting large F1/accuracy gains.

Does not appear to operationalize or measure explanation quality (usefulness or faithfulness); evaluation focuses on task metrics and some qualitative/formatting observations about exemplars and patch outputs.

[11] [Context-Enhanced Vulnerability Detection Based on Large Language Model](#)

Yixin Yang, ..., and Hailong Sun. ArXiv, 2025. 4 citations.

38% Topic Match

Proposes a context-enhanced LLM vulnerability-detection approach using abstracted program analyses. Combines inter-procedural control/data-flow abstractions with source code, feeds varying-granularity context and prompts (CoT, few-shot) to GPT-4, DeepSeek, CodeLLaMA, and measures detection accuracy/false positives. Uses explain-before-answer style CoT examples (ask for code analysis then verdict), but does not report explicit operationalized evaluations of explanation quality (usefulness or faithfulness).

[12] [LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning](#)

Yuqiang Sun, ..., and Yang Liu. ArXiv, 2024. 84 citations.

29% Topic Match

Proposes a unified evaluation framework (LLM4Vuln) to decouple LLM vulnerability reasoning from external aids. Implements UniVul benchmark (Solidity/Java/C/C++), tests six LLMs on 147 vuln + 147 non-vuln examples, and collects structured outputs: label + natural-language rationale via function-calling. Evaluates multiple prompt schemes (including CoT and Post CoT), tool/knowledge augmentation, and uses GPT-4 to auto-structure/annotate rationales — but does not state whether it measures rationale usefulness/faithfulness with human or behavioral faithfulness tests.

[13] [Attention Is All You Need for LLM-based Code Vulnerability Localization](#)

Yue Li, ..., and Fengyuan Xu. ArXiv, 2024. 10 citations.

18% Topic Match

Demonstrates a comparison of prompting/decoding regimes including explain-before-answer CoT for vulnerability localization. Evaluates LOVA under direct-output, few-shot CoT (explain-before-answer), MoA, and rStar, reporting localization accuracy and attention-map examples. Does not appear to measure explanation quality or faithfulness (no human scoring, causal/perturbation tests, or explicit rationale-overlap metrics); focuses on final localization performance and attention visualizations.

[14] [If LLMs Would Just Look: Simple Line-by-line Checking Improves Vulnerability Localization](#)

Yue Li, ..., and Sheng Zhong. ArXiv, 2024. 3 citations.

10% Topic Match

Demonstrates a line-level attention-based method (LOVA) to improve vulnerability localization. Tracks LLM decoder self-attention per line and uses line-by-line checking; compares localization under four prompting regimes including a few-shot explain-before-answer CoT baseline. Does not appear to operationalize or evaluate explanation quality (usefulness/faithfulness) or perform behavioral faithfulness tests; CoT is only a baseline rather than the paper's focus.

[15] [Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities](#)

Avishree Khare, ..., and Mayur Naik. 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), 2023. 74 citations.

8% Topic Match

Evaluates LLM vulnerability detection performance under multiple prompting styles, including explain-before-answer CoT. Tests 16 LLMs on 5,000 code samples across five datasets (Java/C/C++), comparing prompts: dataflow, CoT (reasoning first), and dataflow+self-reflection. Relevant because it enforces rationale-first CoT and measures task metrics by prompt, but it does not operationalize or quantitatively evaluate explanation quality (no localization/faithfulness/usefulness metrics).

[16] [Exploration On Prompting LLM With Code-Specific Information For Vulnerability Detection](#)

Zhihong Liu, ..., and Qing Liao. 2024 IEEE International Conference on Software Services Engineering (SSE), 2024. 6 citations.

5% Topic Match

Analyzes the effect of different GPT prompting styles for vulnerability detection. Compares basic prompts, prompts augmented with code-specific info (similar code, data-flow graphs), and chain-of-thought (CoT) prompts; experiments vary info placement and temperature. Notes: evaluates CoT (rationale-style) prompting but paper appears to focus on task accuracy improvements; unclear whether it enforces an explain-before-answer protocol or measures explanation usefulness/faithfulness (no mention of explicit explanation-quality or localization metrics).

[17] [Beyond ChatGPT: Enhancing Software Quality Assurance Tasks with Diverse LLMs and Validation Techniques](#)

Ratnadira Widyasari, ..., and Lizi Liao. ArXiv, 2024. 9 citations.

4% Topic Match

Abstract: With the advancement of Large Language Models (LLMs), their application in Software Quality Assurance (SQA) has increased. However, the current focus of these applications is predominantly on ChatGPT. There remains a gap in understanding the performance of various LLMs in this critical domain. This paper aims to address this gap by conducting a comprehensive investigation into the capabilities of several LLMs across two SQA tasks: fault localization and vulnerability detection. We conducted comparative studies using GPT-3.5, GPT-4o, and four other publicly available LLMs (LLaMA-3-70B, LLaMA-3-8B, Gemma-7B, and Mixtral-8x7B), to evaluate their effectiveness in these tasks. Our findings reveal that several...

[18] [Prompt-Enhanced Software Vulnerability Detection Using ChatGPT](#)

Chenyuan Zhang, ..., and Hui Li. 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 2023. 85 citations.

3% Topic Match

Abstract: With the increase in software vulnerabilities that cause significant economic and social losses, automatic vulnerability detection has become essential in software development and maintenance. Recently, large language models (LLMs) have received considerable attention due to their stunning intelligence, and some studies consider using ChatGPT for vulnerability detection. However, they do not fully consider the characteristics of LLMs, since their designed questions to ChatGPT are simple without a prompt design tailored for vulnerability detection. This paper launches a study on the performance of software vulnerability detection using ChatGPT with different prompt designs. Firstly, we complement previous work by applying various improvements...

[19] [VulAgent: Hypothesis-Validation based Multi-Agent Vulnerability Detection](#)

Ziliang Wang, ..., and Zhi Jin. ArXiv, 2025. 1 citations.

3% Topic Match

Abstract: The application of language models to project-level vulnerability detection remains challenging, owing to the dual requirement of accurately localizing security-sensitive code and correctly correlating and reasoning over complex program context. We present VulAgent, a multi-agent vulnerability detection framework based on hypothesis validation. Our design is inspired by how human auditors review code: when noticing a sensitive operation, they form a hypothesis about a possible vulnerability, consider potential trigger paths, and then verify the hypothesis against the surrounding context. VulAgent implements a semantics-sensitive, multi-view detection pipeline: specialized agents, each aligned to a specific analysis perspective (e.g., memory, authorization), collaboratively surface and...

[20] [Detecting Bugs with Substantial Monetary Consequences by LLM and Rule-based Reasoning](#)

Brian Zhang and Zhuo Zhang. Advances in Neural Information Processing Systems 37, 2024. 4 citations.

2% Topic Match

Abstract: Financial transactions are increasingly being handled by automated programs called smart contracts. However, one challenge in the adaptation of smart contracts is the presence of vulnerabilities, which can cause significant monetary loss. In 2024, \$247.88 M was lost in 20 smart contract exploits. According to a recent study, accounting bugs (i.e., incorrect implementations of domain-specific financial models) are the most prevalent type of vulnerability, and are one of the most difficult to find, requiring substantial human efforts. While Large Language Models (LLMs) have shown promise in identifying these bugs, they often suffer from lack of generalization of vulnerability types...

- [21] [Manual Prompt Engineering is Not Dead: A Case Study on Large Language Models for Code Vulnerability Detection with DSPy](#)
Fouad Trad and Ali Chehab. 2025 8th International Conference on Data Science and Machine Learning Applications (CDMA), 2025. 3 citations.
2% Topic Match
Abstract: Automated prompt engineering tools have recently emerged as a promising solution to simplify the traditional manual task of crafting prompts for large language models (LLMs). This study investigates whether such tools can fully replace manual prompt engineering for code vulnerability detection. We leverage the DSPy (Declarative Self-improving Python) framework, which uses modular "signatures" rather than prompts to specify tasks. In DSPy, a signature defines the expected input-output behavior of a task and can optionally include a description to guide the model's objective. This signature is then automatically translated into an optimized prompt through DSPy's modules and optimizers. This study compares...
- [22] [DLAP: A Deep Learning Augmented Large Language Model Prompting Framework for Software Vulnerability Detection](#)
Yanjing Yang, ..., and He Zhang. J. Syst. Softw., 2024. 25 citations.
2% Topic Match
Abstract: Software vulnerability detection is generally supported by automated static analysis tools, which have recently been reinforced by deep learning (DL) models. However, despite the superior performance of DL-based approaches over rule-based ones in research, applying DL approaches to software vulnerability detection in practice remains a challenge due to the complex structure of source code, the black-box nature of DL, and the domain knowledge required to understand and validate the black-box results for addressing tasks after detection. Conventional DL models are trained by specific projects and, hence, excel in identifying vulnerabilities in these projects but not in others. These models with...
- [23] [LLM-SmartAudit: Advanced Smart Contract Vulnerability Detection](#)
Zhiyuan Wei, ..., and Xianhao Zhang. ArXiv, 2024. 22 citations.
1% Topic Match
Abstract: The immutable nature of blockchain technology, while revolutionary, introduces significant security challenges, particularly in smart contracts. These security issues can lead to substantial financial losses. Current tools and approaches often focus on specific types of vulnerabilities. However, a comprehensive tool capable of detecting a wide range of vulnerabilities with high accuracy is lacking. This paper introduces LLM-SmartAudit, a novel framework leveraging the advanced capabilities of Large Language Models (LLMs) to detect and analyze vulnerabilities in smart contracts. Using a multi-agent conversational approach, LLM-SmartAudit employs a collaborative system with specialized agents to enhance the audit process. To evaluate the effectiveness of...
- [24] [AutoPatch: Multi-Agent Framework for Patching Real-World CVE Vulnerabilities](#)
Minjae Seo, ..., and Seungwon Shin. ArXiv, 2025. 1 citations.
1% Topic Match
Abstract: Large Language Models (LLMs) have emerged as promising tools in software development, enabling automated code generation and analysis. However, their knowledge is limited to a fixed cutoff date, making them prone to generating code vulnerable to newly disclosed CVEs. Frequent fine-tuning with new CVE sets is costly, and existing LLM-based approaches focus on oversimplified CVE examples and require providing explicit bug locations to LLMs, limiting their ability to patch complex real-world vulnerabilities. To address these limitations, we propose AutoPatch, a multi-agent framework designed to patch vulnerable LLM-generated code, particularly those introduced after the LLMs' knowledge cutoff. AutoPatch integrates Retrieval-Augmented Generation...
- [25] [Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study](#)
Karl Tamberg and Hayretin Bah_iIEEE Access, 2024. 26 citations.
1% Topic Match
Abstract: Despite various approaches being employed to detect software vulnerabilities, the number of reported software vulnerabilities shows an upward trend over the years. This suggests the problems are not caught before the code is released, which could be caused by many factors, like lack of awareness, limited efficacy of the existing vulnerability detection tools or the tools not being user-friendly. To help combat some issues with traditional vulnerability detection tools, we propose using large language models (LLMs) to assist in finding vulnerabilities in source code. LLMs have shown a remarkable ability to understand and generate code, underlining their potential in code-related...
- [26] [Detecting Code Vulnerabilities using LLMs](#)
Larry Huynh, ..., and Jin B. Hong. 2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), 2025. 2 citations.
1% Topic Match
Abstract: Large language models (LLMs) have emerged as a promising tool for detecting code vulnerabilities, potentially offering advantages over traditional rule-based methods. This paper proposes an enhanced framework for vulnerability detection using LLMs, incorporating various prompt engineering strategies to improve performance. We evaluate several techniques, including role-based prompting, zero-shot chain-of-thought, and structured prompting approaches, on the DiverseVul dataset of C/C++ vulnerabilities. Our experiments assess the framework's performance across different code structures, contextual information levels, and LLM capabilities. Our results show that using our dynamic prompt engineering technique, you can improve the F1 score by up to 100% with GPT-3.5, a widely...
- [27] [Vulnerability Detection with Code Language Models: How Far are We?](#)
Yanguibo Ding, ..., and Yizheng Chen. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2024. 122 citations.
1% Topic Match
Abstract: In the context of the rising interest in code language models (code LMs) and vulnerability detection, we study the effectiveness of code LMs for detecting vulnerabilities. Our analysis reveals significant shortcomings in existing vulnerability datasets, including poor data quality, low label accuracy, and high duplication rates, leading to unreliable model performance in realistic vulnerability detection scenarios. Additionally, the evaluation methods used with these datasets are not representative of real-world vulnerability detection. To address these challenges, we introduce Primevul, a new dataset for training and evaluating code LMs for vulnerability detection. Primevul incorporates a novel set of data labeling techniques that...
- [28] [Question Decomposition Improves the Faithfulness of Model-Generated Reasoning](#)
Ansh Radhakrishnan, ..., and Ethan Perez. ArXiv, 2023. 101 citations.
1% Topic Match
Abstract: As large language models (LLMs) perform more difficult tasks, it becomes harder to verify the correctness and safety of their behavior. One approach to help with this issue is to prompt LLMs to externalize their reasoning, e.g., by having them generate step-by-step reasoning as they answer a question (Chain-of-Thought; CoT). The reasoning may enable us to check the process that models use to perform tasks. However, this approach relies on the stated reasoning faithfully reflecting the model's actual reasoning, which is not always the case. To improve over the faithfulness of CoT reasoning, we have models generate reasoning by decomposing...
- [29] [LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights](#)
Ze Sheng, ..., and Jeff Huang. ACM Computing Surveys, 2025. 14 citations.
1% Topic Match
Abstract: Large Language Models (LLMs) are emerging as transformative tools for software vulnerability detection. Traditional methods, including static and dynamic analysis, face limitations in efficiency, false-positive rates, and scalability with modern software complexity. Through code structure analysis, pattern identification, and repair suggestion generation, LLMs demonstrate a novel approach to vulnerability mitigation. This survey examines LLMs in vulnerability detection, analyzing problem formulation, model selection, application methodologies, datasets, and evaluation metrics. We investigate current research challenges, emphasizing cross-language detection, multimodal integration, and repository-level analysis. Based on our findings, we propose solutions addressing dataset scalability, model interpretability, and low-resource scenarios. Our contributions include: (1)...
- [30] [VulSolver: Vulnerability Detection via LLM-Driven Constraint Solving](#)
Xiang Li, ..., and Yuanchao Zhang. ArXiv, 2025. 0 citations.
1% Topic Match
Abstract: Traditional vulnerability detection methods rely heavily on predefined rule matching, which often fails to capture vulnerabilities accurately. With the rise of large language models (LLMs), leveraging their ability to understand code semantics has emerged as a promising direction for achieving more accurate and efficient vulnerability detection. However, current LLM-based approaches face significant challenges: instability in model outputs, limitations in context length, and hallucination. As a result, many existing solutions either use LLMs merely to enrich predefined rule sets, thereby keeping the detection process fundamentally rule-based, or over-rely on them, leading to poor robustness. To address these challenges, we propose a...
- [31] [Large language models replicate and predict human cooperation across experiments in game theory](#)

Andrea Cera Palatsi, ..., and Max Pellert. ArXiv, 2025. 0 citations.

1% Topic Match

Abstract: Large language models (LLMs) are increasingly used both to make decisions in domains such as health, education and law, and to simulate human behavior. Yet how closely LLMs mirror actual human decision-making remains poorly understood. This gap is critical: misalignment could produce harmful outcomes in practical applications, while failure to replicate human behavior renders LLMs ineffective for social simulations. Here, we address this gap by developing a digital twin of game-theoretic experiments and introducing a systematic prompting and probing framework for machine-behavioral evaluation. Testing three open-source models (Llama, Mistral and Qwen), we find that Llama reproduces human cooperation patterns with...

[32] [R2Vul: Learning to Reason about Software Vulnerabilities with Reinforcement Learning and Structured Reasoning Distillation](#)

M. Weyssow, ..., and David Lo. ArXiv, 2025. 4 citations.

0% Topic Match

Abstract: Large language models (LLMs) have shown promising performance in software vulnerability detection, yet their reasoning capabilities remain unreliable. We propose R2Vul, a method that combines reinforcement learning from AI feedback (RLAIF) and structured reasoning distillation to teach small code LLMs to detect vulnerabilities while generating security-aware explanations. Unlike prior chain-of-thought and instruction tuning approaches, R2Vul rewards well-founded over deceptively plausible vulnerability explanations through RLAIF, which results in more precise detection and high-quality reasoning generation. To support RLAIF, we construct the first multilingual preference dataset for vulnerability detection, comprising 18,000 high-quality samples in C#, JavaScript, Java, Python, and C. We evaluate...

[33] [SCCA: A Multi-Agent Code Security Analysis Framework for AI-Assisted Code Generation](#)

Rui Jiao, ..., and Boyang Ma. 2025 IEEE 22nd International Conference on Mobile Ad-Hoc and Smart Systems (MASS), 2025. 0 citations.

0% Topic Match

Abstract: This paper presents SCCA, a novel multi-agent security analysis framework for AI-assisted code generation environments. Our system combines three specialized agents—AST-based structural analysis, LLM-enhanced vulnerability detection, and data flow security assessment—to overcome limitations of traditional security tools. We evaluate the framework using different LLM configurations (e.g., Claude-4, GPT-4o) across diverse project types, demonstrating the impact of LLM selection on the quality of security analysis. Results show our framework with Claude-4 achieves superior performance in vulnerability detection and explanation quality, with the multi-agent approach significantly outperforming traditional methods. Furthermore, our framework produces structured reports specifically designed for automated remediation, enabling high...

[34] [Boosting Vulnerability Detection of LLMs via Curriculum Preference Optimization with Synthetic Reasoning Data](#)

Xinjie Wen, ..., and Deheng Ye. Unknown journal, 2025. 5 citations.

0% Topic Match

Abstract: Large language models (LLMs) demonstrate considerable proficiency in numerous coding-related tasks; however, their capabilities in detecting software vulnerabilities remain limited. This limitation primarily stems from two factors: (1) the absence of reasoning data related to vulnerabilities, which hinders the models' ability to capture underlying vulnerability patterns; and (2) their focus on learning semantic representations rather than the reason behind them, thus failing to recognize semantically similar vulnerability samples. Furthermore, the development of LLMs specialized in vulnerability detection is challenging, particularly in environments characterized by the scarcity of high-quality datasets. In this paper, we propose a novel framework ReVD that excels...

[35] [SoK: Towards Effective Automated Vulnerability Repair](#)

Ying Li, ..., and Yuan Tian. ArXiv, 2025. 4 citations.

0% Topic Match

Abstract: The increasing prevalence of software vulnerabilities necessitates automated vulnerability repair (AVR) techniques. This Systematization of Knowledge (SoK) provides a comprehensive overview of the AVR landscape, encompassing both synthetic and real-world vulnerabilities. Through a systematic literature review and quantitative benchmarking across diverse datasets, methods, and strategies, we establish a taxonomy of existing AVR methodologies, categorizing them into template-guided, search-based, constraint-based, and learning-driven approaches. We evaluate the strengths and limitations of these approaches, highlighting common challenges and practical implications. Our comprehensive analysis of existing AVR methods reveals a diverse landscape with no single "best" approach. Learning-based methods excel in specific scenarios but...

[36] [Chain of Thought Prompting Elicits Reasoning in Large Language Models](#)

Jason Wei, ..., and Denny Zhou. ArXiv, 2022. 13395 citations.

0% Topic Match

Abstract: We explore how generating a chain of thought -- a series of intermediate reasoning steps -- significantly improves the ability of large language models to perform complex reasoning. In particular, we show how such reasoning abilities emerge naturally in sufficiently large language models via a simple method called chain of thought prompting, where a few chain of thought demonstrations are provided as exemplars in prompting. Experiments on three large language models show that chain of thought prompting improves performance on a range of arithmetic, commonsense, and symbolic reasoning tasks. The empirical gains can be striking. For instance, prompting a 540B-parameter...

[37] [Can LLM Prompting Serve as a Proxy for Static Analysis in Vulnerability Detection](#)

Ira Ceka, ..., and Baishakhi Ray. ArXiv, 2024. 3 citations.

0% Topic Match

Abstract: Despite their remarkable success, large language models (LLMs) have shown limited ability on safety-critical code tasks such as vulnerability detection. Typically, static analysis (SA) tools, like CodeQL, CodeGuru Security, etc., are used for vulnerability detection. SA relies on predefined, manually-crafted rules for flagging various vulnerabilities. Thus, effectiveness of SA in detecting vulnerabilities depends on human experts and is known to report high error rates. In this study we investigate whether LLM prompting can be an effective alternative to these static analyzers in the partial code setting. We propose prompting strategies that integrate natural language instructions of vulnerabilities with contrastive chain-of-thought...

[38] [Adapting LLM to Generate Vulnerability Descriptions and Repair Suggestions](#)

Wenxin Tao, ..., and Xiaohong Su. 2025 IEEE 5th International Conference on Software Engineering and Artificial Intelligence (SEAI), 2025. 0 citations.

0% Topic Match

Abstract: Software vulnerabilities pose serious threats to the security of modern software systems. Existing detection tools can flag or pinpoint vulnerability but offer little guidance on how to fix them. In this paper, we leverage the generative and reasoning strengths of LLMs to bridge this gap, combining prompt tuning, retrieval-augmented generation, and chain-of-thought techniques to produce both clear vulnerability descriptions and concrete repair suggestions. We evaluated five mainstream LLMs on the BigVul dataset, and the combination of RAG and CoT methods achieves leading performance across multiple metrics in both the vulnerability description generation and repair suggestion generation tasks.

[39] [Fine-Tuning LLMs to Analyze Multiple Dimensions of Code Review: A Maximum Entropy Regulated Long Chain-of-Thought Approach](#)

Yongda Yu, ..., and Guoping Rong. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have shown great potential in supporting automated code review due to their impressive capabilities in context understanding and reasoning. However, these capabilities are still limited compared to human-level cognition because they are heavily influenced by the training data. Recent research has demonstrated significantly improved performance through fine-tuning LLMs with code review data. However, compared to human reviewers who often simultaneously analyze multiple dimensions of code review to better identify issues, the full potential of these methods is hampered by the limited or vague information used to fine-tune the models. This paper contributes MelcotCR, a chain-of-thought (COT)...

[40] [Distilling Algorithmic Reasoning from LLMs via Explaining Solution Programs](#)

Jierui Li and Raymond Mooney. ArXiv, 2024. 3 citations.

0% Topic Match

Abstract: Distilling explicit chain-of-thought reasoning paths has emerged as an effective method for improving the reasoning abilities of large language models (LLMs) across various tasks. However, when tackling complex tasks that pose significant challenges for state-of-the-art models, this technique often struggles to produce effective chains of thought that lead to correct answers. In this work, we propose a novel approach to distill reasoning abilities from LLMs by leveraging

their capacity to explain solutions. We apply our method to solving competitive-level programming challenges. More specifically, we employ an LLM to generate explanations for a set of pairs, then use pairs to fine-tune a smaller...

[41] [Large Language Model for Vulnerability Detection: Emerging Results and Future Directions](#)

Xin Zhou, ..., and David Lo. 2024 IEEE/ACM 46th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), 2024. 98 citations.

0% Topic Match

Abstract: Previous learning-based vulnerability detection methods relied on either medium-sized pretrained models or smaller neural networks from scratch. Recent advancements in Large Pre-Trained Language Models (LLMs) have showcased remarkable few-shot learning capabilities in various tasks. However, the effectiveness of LLMs in detecting software vulnerabilities is largely unexplored. This paper aims to bridge this gap by exploring how LLMs perform with various prompts, particularly focusing on two state-of-the-art LLMs: GPT-3.5 and GPT-4. Our experimental results showed that GPT-3.5 achieves competitive performance with the prior state-of-the-art vulnerability detection approach and GPT-4 consistently outperformed the state-of-the-art.

[42] [When LLMs Copy to Think: Uncovering Copy-Guided Attacks in Reasoning LLMs](#)

Yue Li, ..., and Sheng Zhong. 2025 IEEE 22nd International Conference on Mobile Ad-Hoc and Smart Systems (MASS), 2025. 1 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have become integral to automated code analysis, enabling tasks such as vulnerability detection and code comprehension. However, their integration introduces novel attack surfaces. In this paper, we identify and investigate a new class of prompt-based attacks, termed Copy-Guided Attacks (CGA), which exploit the inherent copying tendencies of reasoning-capable LLMs. By injecting carefully crafted triggers into external code snippets, adversaries can induce the model to replicate malicious content during inference. This behavior enables two classes of vulnerabilities: inference length manipulation, where the model generates abnormally short or excessively long reasoning traces; and inference result manipulation, where the...

[43] [A Study on Thinking Patterns of Large Reasoning Models in Code Generation](#)

Kevin Halim, ..., and Yang Liu. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Currently, many large language models (LLMs) are utilized for software engineering tasks such as code generation. The emergence of more advanced models known as large reasoning models (LRMs), such as OpenAI's o3, DeepSeek R1, and Qwen3. They have demonstrated the capability of performing multi-step reasoning. Despite the advancement in LRMs, little attention has been paid to systematically analyzing the reasoning patterns these models exhibit and how such patterns influence the generated code. This paper presents a comprehensive study aimed at investigating and uncovering the reasoning behavior of LRMs during code generation. We prompted several state-of-the-art LRMs of varying sizes with...

[44] [Towards Reasoning in Large Language Models: A Survey](#)

Jie Huang and K. Chang. ArXiv, 2022. 766 citations.

0% Topic Match

Abstract: Reasoning is a fundamental aspect of human intelligence that plays a crucial role in activities such as problem solving, decision making, and critical thinking. In recent years, large language models (LLMs) have made significant progress in natural language processing, and there is observation that these models may exhibit reasoning abilities when they are sufficiently large. However, it is not yet clear to what extent LLMs are capable of reasoning. This paper provides a comprehensive overview of the current state of knowledge on reasoning in LLMs, including techniques for improving and eliciting reasoning in these models, methods and benchmarks for evaluating...

[45] [Larger Is Not Always Better: Exploring Small Open-source Language Models in Logging Statement Generation](#)

Renyi Zhong, ..., and Michael R. Lyu. ACM Transactions on Software Engineering and Methodology, 2025. 1 citations.

0% Topic Match

Abstract: Developers use logging statements to create logs that document system behavior and aid in software maintenance. As such, high-quality logging is essential for effective maintenance; however, manual logging often leads to errors and inconsistency. Recent methods emphasize using large language models (LLMs) for automated logging statement generation, but these present privacy and resource issues, hindering their suitability for enterprise use. This paper presents the first large-scale empirical study evaluating small open-source language models (SOLMs) for automated logging statement generation. We evaluate four prominent SOLMs using various prompt strategies and parameter-efficient fine-tuning techniques, such as Low-Rank Adaptation (LoRA) and Retrieval-Augmented Generation...

[46] [GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning](#)

Guilong Lu, ..., and Zhilong Cai. J. Syst. Softw., 2024. 132 citations.

0% Topic Match

No summary or abstract available

[47] [Layered Chain-of-Thought Prompting for Multi-Agent LLM Systems: A Comprehensive Approach to Explainable Large Language Models](#)

Manish Sanwal. ArXiv, 2025. 8 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) leverage chain-of-thought (CoT) prompting to provide step-by-step rationales, improving performance on complex tasks. Despite its benefits, vanilla CoT often fails to fully verify intermediate inferences and can produce misleading explanations. In this work, we propose Layered Chain-of-Thought (Layered-CoT) Prompting, a novel framework that systematically segments the reasoning process into multiple layers, each subjected to external checks and optional user feedback. We expand on the key concepts, present three scenarios -- medical triage, financial risk assessment, and agile engineering -- and demonstrate how Layered-CoT surpasses vanilla CoT in terms of transparency, correctness, and user engagement. By integrating...

[48] [Knowledge-Aware Code Generation with Large Language Models](#)

Tao Huang, ..., and Chen Lyu. 2024 IEEE/ACM 32nd International Conference on Program Comprehension (ICPC), 2024. 13 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) perform well on basic programming problems. However, they encounter challenges when dealing with complex tasks involving the use of diverse algorithmic and data structure skills, particularly programming competition-level problems. Notably, ChatGPT exhibits proficient performance on problems it has encountered during its pre-training phase, but this performance deteriorates when faced with novel problems. Consequently, enhancing the ability of LLMs to address unfamiliar problems has emerged as a pivotal research focus. The problem-solving process of LLMs mirrors human programmers' approach to a certain extent. When confronted with new programming tasks, human programmers engage in task planning and code...

[49] [Assessing the Effectiveness of Vulnerability Detection via Prompt Tuning: An Empirical Study](#)

Guilong Lu, ..., and Hao Shen. 2023 30th Asia-Pacific Software Engineering Conference (APSEC), 2023. 12 citations.

0% Topic Match

Abstract: In vulnerability detection approaches based on deep learning, fine-tuning with Pre-trained Language Models (PLMs) is a prevalent technique. Unfortunately, a natural gap exists between model pre-training tasks and vulnerability detection tasks due to different input formats, and the performance of fine-tuning relies on downstream dataset scales. Recently, prompt tuning has been used to alleviate these issues. However, it has not received enough attention in vulnerability detection. To assess the effectiveness of prompt tuning, we consider three classical vulnerability detection tasks: within-domain vulnerability detection, cross-domain vulnerability detection, and vulnerability type detection. Our empirical study considers three popular PLMs: CodeBERT, CodeT5, and...

[50] [A New Approach to Web Application Security: Utilizing GPT Language Models for Source Code Inspection](#)

Zoltán Szabó and Vilmos Bilicki. Future Internet, 2023. 31 citations.

0% Topic Match

Abstract: Due to the proliferation of large language models (LLMs) and their widespread use in applications such as ChatGPT, there has been a significant increase in interest in AI over the past year. Multiple researchers have raised the question: how will AI be applied and in what areas? Programming, including the generation, interpretation, analysis, and documentation of static program code based on prompts is one of the most promising fields. With the GPT API, we have explored a new aspect of this: static analysis of the source code of front-end applications at the endpoints of the data path. Our focus was the...

[51] [Vulnerability Handling of AI-Generated Code - Existing Solutions and Open Challenges](#)

Sabrina Kaniewski, ..., and Tobias Heer. 2024 Conference on AI, Science, Engineering, and Technology (AIxSET), 2024. 2 citations.

0% Topic Match

Abstract: The increasing use of generative Artificial Intelligence (AI) in modern software engineering, particularly Large Language Models (LLMs) for code generation, has transformed professional software development by boosting productivity and automating development processes. This adoption, however, has highlighted a significant issue: the introduction of security vulnerabilities into the code. These vulnerabilities result, e.g., from flaws in the training data that propagate into the generated code, creating challenges in tackling them in established ways. Traditional vulnerability handling processes often involve extensive manual review. Applying such traditional processes to AI-generated code is challenging. AI-generated code may include several similar vulnerabilities, possibly in slightly...

[52] [CodeSense: a Real-World Benchmark and Dataset for Code Semantic Reasoning](#)

Monoshi Kumar Roy, ..., and Wei Le. ArXiv, 2025. 3 citations.

0% Topic Match

Abstract: Understanding and reasoning about code semantics is essential for enhancing code LLMs'abilities to solve real-world software engineering (SE) tasks. Although several code reasoning benchmarks exist, most rely on synthetic datasets or educational coding problems and focus on coarse-grained reasoning tasks such as input/output prediction, limiting their effectiveness in evaluating LLMs in practical SE contexts. To bridge this gap, we propose CodeSense, the first benchmark that makes available a spectrum of fine-grained code reasoning tasks concerned with the software engineering of real-world code. We collected Python, C and Java software projects from real-world repositories. We executed tests from these repositories, collected...

[53] [LLM-Assisted Static Analysis for Detecting Security Vulnerabilities](#)

Ziyang Li, ..., and Mayur Naik. ArXiv, 2024. 109 citations.

0% Topic Match

Abstract: Software is prone to security vulnerabilities. Program analysis tools to detect them have limited effectiveness in practice due to their reliance on human labeled specifications. Large language models (or LLMs) have shown impressive code generation capabilities but they cannot do complex reasoning over code to detect such vulnerabilities especially since this task requires whole-repository analysis. We propose IRIS, a neuro-symbolic approach that systematically combines LLMs with static analysis to perform fine-grained code reasoning for security vulnerability detection. Specifically, IRIS leverages LLMs to infer taint specifications and perform contextual analysis, alleviating needs for human specifications and inspection. For evaluation, we curate a...

[54] [Benchmarking LLM for Zero-day Vulnerabilities](#)

Lisha M, ..., and Madhusoodhana Chari S. 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), 2024. 3 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have emerged as powerful tools for natural language processing tasks. This paper investigates the efficacy of various LLMs in detecting zero-day vulnerabilities, crucial for preemptive cybersecurity measures. Through benchmarking and experimentation, we analyze the performance of LLMs in unstructured querying scenarios. Our findings aim to enhance understanding of LLM capabilities and contribute to the advancement of vulnerability detection methodologies.

[55] [Large Language Model for Vulnerability Detection and Repair: Literature Review and Roadmap](#)

Xin Zhou, ..., and †. DavidLo. Journal Not Provided, Unknown year. 13 citations.

0% Topic Match

No summary or abstract available

[56] [Think Like Human Developers: Harnessing Community Knowledge for Structured Code Reasoning](#)

Chengran Yang, ..., and David Lo. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have significantly advanced automated code generation, yet they struggle with complex coding tasks requiring multi-step logical reasoning. High-quality reasoning data is crucial for improving LLMs' reasoning capabilities, but such datasets remain scarce. Existing approaches either rely on computationally expensive reinforcement learning (RL) or error-prone reasoning chains synthesized by LLMs, posing challenges in scalability and accuracy. To address this challenge, we propose SVRC (Structured and Validated Reasoning Chains for Code Generation), a novel framework that mines, restructures, and enriches reasoning chains from community-driven discussions on software engineering platforms. SVRC refines unstructured and incomplete discussions of coding problems...

[57] [Multitask-Based Evaluation of Open-Source LLM on Software Vulnerability](#)

Xin Yin, ..., and Shaohua Wang. IEEE Transactions on Software Engineering, 2024. 54 citations.

0% Topic Match

Abstract: This paper proposes a pipeline for quantitatively evaluating interactive Large Language Models (LLMs) using publicly available datasets. We carry out an extensive technical evaluation of LLMs using Big-Vul covering four different common software vulnerability tasks. This evaluation assesses the multi-tasking capabilities of LLMs based on this dataset. We find that the existing state-of-the-art approaches and pre-trained Language Models (LMs) are generally superior to LLMs in software vulnerability detection. However, in software vulnerability assessment and location, certain LLMs (e.g., CodeLlama and WizardCoder) have demonstrated superior performance compared to pre-trained LMs, and providing more contextual information can enhance the vulnerability assessment capabilities...

[58] [How to Explain a Patch: An Empirical Study of Patch Explanations in Open Source Projects](#)

Jingjing Liang, ..., and Gang Huang. 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE), 2019. 9 citations.

0% Topic Match

Abstract: Abstract-Bugs are inevitable in software development and maintenance processes. Recently a lot of research efforts have been devoted to automatic program repair, aiming to reduce the efforts of debugging. However, since it is difficult to ensure that the generated patches meet all quality requirements such as correctness, developers still need to review the patch. In addition, current techniques produce only patches without explanation, making it difficult for the developers to understand the patch. Therefore, we believe a more desirable approach should generate not only the patch but also an explanation of the patch. To generate a patch explanation, it is...

[59] [Benchmarking Prompt Engineering Techniques for Secure Code Generation with GPT Models](#)

Marc Bruni, ..., and Martin Kropp. 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), 2025. 11 citations.

0% Topic Match

Abstract: Prompt engineering reduces reasoning mistakes in Large Language Models (LLMs). However, its effectiveness in mitigating vulnerabilities in LLM-generated code remains underexplored. To address this gap, we implemented a benchmark to automatically assess the impact of various prompt engineering strategies on code security. Our benchmark leverages two peer-reviewed prompt datasets and employs static scanners to evaluate code security at scale. We tested multiple prompt engineering techniques on GPT-3.5-turbo, GPT-4o, and GPT-4o-mini. Our results show that for GPT-4o and GPT-4o-mini, a security-focused prompt prefix can reduce the occurrence of security vulnerabilities by up to 56%. Additionally, all tested models demonstrated the ability...

[60] [A Preliminary Study on Using Large Language Models in Software Pentesting](#)

Kumar Shashwat, ..., and Armin Ziaie Tabari. ArXiv, 2024. 8 citations.

0% Topic Match

Abstract: Large language models (LLM) are perceived to offer promising potentials for automating security tasks, such as those found in security operation centers (SOCs). As a first step towards evaluating this perceived potential, we investigate the use of LLMs in software pentesting, where the main task is to automatically identify software security vulnerabilities in source code. We hypothesize that an LLM-based AI agent can be improved over time for a specific security task as human operators interact with it. Such improvement can be made, as a first step, by engineering prompts fed to the LLM based on the responses produced, to...

[61] [Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems](#)

Zhiyuan Wei, ..., and Liehuang Zhu. IEEE Transactions on Software Engineering, 2025. 5 citations.

0% Topic Match

No summary or abstract available

[62] [LLaMA-Reviewer: Advancing Code Review Automation with Large Language Models through Parameter-Efficient Fine-Tuning](#)

Jun Lu, ..., and Chun Zuo. 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), 2023. 96 citations.

0% Topic Match

Abstract: The automation of code review activities, a long-standing pursuit in software engineering, has been primarily addressed by numerous domain-specific pre-trained models. Despite their success, these models frequently demand extensive resources for pre-training from scratch. In contrast, Large Language Models (LLMs) provide an intriguing alternative, given their remarkable capabilities when supplemented with domain-specific knowledge. However, their potential for automating code review tasks remains largely unexplored. In response to this research gap, we present LLaMA-Reviewer, an innovative framework that leverages the capabilities of LLaMA, a popular LLM, in the realm of code review. Mindful of resource constraints, this framework employs parameter-efficient fine-tuning (PEFT)...

[63] [How Far Have We Gone in Vulnerability Detection Using Large Language Models](#)

Zeyu Gao, ..., and Chao Zhang. ArXiv, 2023. 37 citations.

0% Topic Match

Abstract: As software becomes increasingly complex and prone to vulnerabilities, automated vulnerability detection is critically important, yet challenging. Given the significant successes of large language models (LLMs) in various tasks, there is growing anticipation of their efficacy in vulnerability detection. However, a quantitative understanding of their potential in vulnerability detection is still missing. To bridge this gap, we introduce a comprehensive vulnerability benchmark VulBench. This benchmark aggregates high-quality data from a wide range of CTF (Capture-the-Flag) challenges and real-world applications, with annotations for each vulnerable function detailing the vulnerability type and its root cause. Through our experiments encompassing 16 LLMs and...

[64] [Rethinking Kernel Program Repair: Benchmarking and Enhancing LLMs with RGym](#)

Kareem Shehata, ..., and Zhiyun Qian. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have revolutionized automated program repair (APR) but current benchmarks like SWE-Bench predominantly focus on userspace applications and overlook the complexities of kernel-space debugging and repair. The Linux kernel poses unique challenges due to its monolithic structure, concurrency, and low-level hardware interactions. Prior efforts such as KGym and CrashFixer have highlighted the difficulty of APR in this domain, reporting low success rates or relying on costly and complex pipelines and pricey cloud infrastructure. In this work, we introduce RGym, a lightweight, platform-agnostic APR evaluation framework for the Linux kernel designed to operate on local commodity hardware. Built...

[65] [Software Vulnerability Detection Using LLM: Does Additional Information Help?](#)

Samiha Shimmi, ..., and Mona Rahimi. 2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops), 2024. 2 citations.

0% Topic Match

Abstract: Unlike conventional machine learning (ML) or deep learning (DL) methods, Large Language Models (LLM) possess the ability to tackle complex tasks through intricate chains of reasoning, a facet often overlooked in existing work on vulnerability detection. Nevertheless, these models have demonstrated variable performance when presented with different prompts (inputs), motivating a surge of research into prompt engineering – the process of optimizing prompts to enhance their performance. This paper studies different prompt settings (zero-shot and few-shot) when using LLMs for software vulnerability detection. Our exploration involves harnessing the power of both natural language (NL) unimodal and NL-PL (programming language) bimodal...

[66] [Can Large Language Models Find And Fix Vulnerable Software?](#)

David A. Noever. ArXiv, 2023. 31 citations.

0% Topic Match

Abstract: In this study, we evaluated the capability of Large Language Models (LLMs), particularly OpenAI's GPT-4, in detecting software vulnerabilities, comparing their performance against traditional static code analyzers like Snyk and Fortify. Our analysis covered numerous repositories, including those from NASA and the Department of Defense. GPT-4 identified approximately four times the vulnerabilities than its counterparts. Furthermore, it provided viable fixes for each vulnerability, demonstrating a low rate of false positives. Our tests encompassed 129 code samples across eight programming languages, revealing the highest vulnerabilities in PHP and JavaScript. GPT-4's code corrections led to a 90% reduction in vulnerabilities, requiring only...

[67] [RealVul: Can We Detect Vulnerabilities in Web Applications with LLM?](#)

Di Cao, ..., and Xiuwei Shang. Unknown journal, 2024. 4 citations.

0% Topic Match

Abstract: The latest advancements in large language models (LLMs) have sparked interest in their potential for software vulnerability detection. However, there is currently a lack of research specifically focused on vulnerabilities in the PHP language, and challenges in data sampling and processing persist, hindering the model's ability to effectively capture the characteristics of specific vulnerabilities. In this paper, we present RealVul, the first LLM-based framework designed for PHP vulnerability detection, addressing these issues. By improving code sampling methods and employing normalization techniques, we can isolate potential vulnerability triggers while streamlining the code and eliminating unnecessary semantic information, enabling the model to...

[68] [Do advanced language models eliminate the need for prompt engineering in software engineering?](#)

Guoqing Wang, ..., and Dan Hao. ACM Transactions on Software Engineering and Methodology, 2024. 12 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have significantly advanced software engineering (SE) tasks, with prompt engineering techniques enhancing their performance in code-related areas. However, the rapid development of foundational LLMs such as the non-reasoning models (GPT-4o and Claude 3.5 Sonnet) and the reasoning model o1 raises questions about the continued effectiveness of these prompt engineering techniques. This paper presents an extensive empirical study that reevaluates various prompt engineering techniques within the context of these advanced LLMs. Focusing on five representative SE tasks, i.e., code generation, code translation, program repair, code summarization, and commit message generation, we assess whether prompt engineering techniques still...

[69] [Examining Zero-Shot Vulnerability Repair with Large Language Models](#)

H. Pearce, ..., and Brendan Dolan-Gavitt. 2023 IEEE Symposium on Security and Privacy (SP), 2021. 260 citations.

0% Topic Match

Abstract: Human developers can produce code with cybersecurity bugs. Can emerging 'smart' code completion tools help repair those bugs? In this work, we examine the use of large language models (LLMs) for code (such as OpenAI's Codex and AI21's Jurassic J-1) for zero-shot vulnerability repair. We investigate challenges in the design of prompts that coax LLMs into generating repaired versions of insecure code. This is difficult due to the numerous ways to phrase key information—both semantically and syntactically—with natural languages. We perform a large scale study of five commercially available, black-box, "off-the-shelf" LLMs, as well as an open-source model and...

[70] [Exploring Prompt Patterns for Effective Vulnerability Repair in Real-World Code by Large Language Models](#)

Yining Luo, ..., and Peng Liu. Proceedings of the 2025 ACM International Workshop on Security and Privacy Analytics, 2025. 0 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have shown promise in automating code vulnerability repair, but their effectiveness in handling real-world code remains limited. This paper investigates the capability of LLMs, in repairing vulnerabilities and proposes a systematic approach to enhance their performance through specialized prompt engineering. Through extensive evaluation of 5,826 code samples, we found that while LLMs successfully repair vulnerabilities in simple cases, they struggle with complex real-world code that involves intricate dependencies, contextual requirements, and multi-file interactions. To address these limitations, we first incorporated Control Flow Graphs (CFGs) as supplementary prompts, achieving a 14.4% success rate in fixing previously unresolvable...

[71] [Explicit Vulnerability Generation with LLMs: An Investigation Beyond Adversarial Attacks](#)

Emir Bosnak, ..., and Anil Koyuncu. 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2025. 0 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) are increasingly used as code assistants, yet their behavior when explicitly asked to generate insecure code remains poorly understood. While prior research has focused on unintended vulnerabilities, this study examines a more direct threat: open-source LLMs generating vulnerable code when prompted. We propose a dual experimental design: (1) Dynamic Prompting, which systematically varies vulnerability type, user persona, and prompt phrasing across structured templates; and (2) Reverse Prompting, which derives natural-language prompts from real vulnerable code samples. We evaluate three open-source 7B-parameter models (Qwen2, Mistral, Gemma) using static analysis to assess both the presence and correctness of...

[72] [A Systematic Review on Detection, Repair, and Explanation of Vulnerabilities in Source Code Using Large Language Models](#)

Lucas B. Germano, ..., and J. C. Duarte. IEEE Access, 2025. 0 citations.

0% Topic Match

Abstract: Software vulnerabilities pose critical risks to the security and reliability of modern systems, requiring effective detection, repair, and explanation techniques. Large Language Models (LLMs) have recently emerged as promising tools for these tasks, yet research in this area remains uneven. This systematic review analyzes 208 peer-reviewed studies published between 2018 and August 2025, examining how LLMs are applied to source code vulnerability analysis. The findings reveal a strong emphasis on vulnerability detection (91.3% of studies), while repair (11.1%) and explanation (5.3%) remain underexplored. Programming language coverage shows similar imbalances: C/C++ is the most frequently studied, whereas widely adopted languages such...

[73] [DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection](#)

Yizheng Chen, ..., and David A. Wagner. Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, 2023. 218 citations.

0% Topic Match

Abstract: We propose and release a new vulnerable source code dataset. We curate the dataset by crawling security issue websites, extracting vulnerability-fixing commits and source codes from the corresponding projects. Our new dataset contains 18,945 vulnerable functions spanning 150 CWEs and 330,492 non-vulnerable functions extracted from 7,514 commits. Our dataset covers 295 more projects than all previous datasets combined. Combining our new dataset with previous datasets, we present an analysis of the challenges and promising research directions of using deep learning for detecting software vulnerabilities. We study 11 model architectures belonging to 4 families. Our results show that deep learning is...

[74] [Evaluating Human Alignment and Model Faithfulness of LLM Rationale](#)

Mohsen Fayyaz, ..., and Nanyun Peng. ArXiv, 2024. 10 citations.

0% Topic Match

Abstract: We study how well large language models (LLMs) explain their generations through rationales -- a set of tokens extracted from the input text that reflect the decision-making process of LLMs. Specifically, we systematically study rationales derived using two approaches: (1) popular prompting-based methods, where prompts are used to guide LLMs in generating rationales, and (2) technical attribution-based methods, which leverage attention or gradients to identify important tokens. Our analysis spans three classification datasets with annotated rationales, encompassing tasks with varying performance levels. While prompting-based self-explanations are widely used, our study reveals that these explanations are not always as "aligned" with the human...

[75] [Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction](#)

Sungmin Kang, ..., and Shin Yoo. 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2022. 240 citations.

0% Topic Match

Abstract: Many automated test generation techniques have been developed to aid developers with writing tests. To facilitate full automation, most existing techniques aim to either increase coverage, or generate exploratory inputs. However, existing test generation techniques largely fall short of achieving more semantic objectives, such as generating tests to reproduce a given bug report. Reproducing bugs is nonetheless important, as our empirical study shows that the number of tests added in open source repositories due to issues was about 28% of the corresponding project test suite size. Meanwhile, due to the difficulties of transforming the expected program semantics in bug reports...

[76] [How Effective Are Neural Networks for Fixing Security Vulnerabilities](#)

Yi Wu, ..., and Sameena Shah. Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023. 114 citations.

0% Topic Match

Abstract: Security vulnerability repair is a difficult task that is in dire need of automation. Two groups of techniques have shown promise: (1) large code language models (LLMs) that have been pre-trained on source code for tasks such as code completion, and (2) automated program repair (APR) techniques that use deep learning (DL) models to automatically fix software bugs. This paper is the first to study and compare Java vulnerability repair capabilities of LLMs and DL-based APR models. The contributions include that we (1) apply and evaluate five LLMs (Codex, CodeGen, CodeT5, PLBART and InCoder), four fine-tuned LLMs, and four DL-based...

[77] [A Mixture of Linear Corrections Generates Secure Code](#)

Weicheng Yu, ..., and Corina S. P. s reanuArXiv, 2025. 1 citations.

0% Topic Match

Abstract: Large language models (LLMs) have become proficient at sophisticated code-generation tasks, yet remain ineffective at reliably detecting or avoiding code vulnerabilities. Does this deficiency stem from insufficient learning about code vulnerabilities, or is it merely a result of ineffective prompting? Using representation engineering techniques, we investigate whether LLMs internally encode the concepts necessary to identify code vulnerabilities. We find that current LLMs encode precise internal representations that distinguish vulnerable from secure code--achieving greater accuracy than standard prompting approaches. Leveraging these vulnerability-sensitive representations, we develop an inference-time steering technique that subtly modulates the model's token-generation probabilities through a mixture of corrections...

[78] [On the Evaluation of Large Language Models in Multilingual Vulnerability Repair](#)

Dong Wang, ..., and Junjie Chen. ACM Transactions on Software Engineering and Methodology, 2025. 0 citations.

0% Topic Match

Abstract: Various Deep Learning-based approaches with pre-trained language models have been proposed for automatically repairing software vulnerabilities. However, these approaches are limited to a specific programming language (C/C++). Recent advances in large language models (LLMs) offer language-agnostic capabilities and strong semantic understanding, exhibiting potential to overcome multilingual vulnerability limitation. Although some work has begun to explore LLM's repair performance, their effectiveness is unsatisfactory. To address these limitations, we conducted a large-scale empirical study to investigate the performance of automated vulnerability repair approaches and state-of-the-art LLMs across seven programming languages. Results show GPT-4o, instruction-tuned with few-shot prompting, performs competitively against the leading...

[79] [LineVul: A Transformer-based Line-Level Vulnerability Prediction](#)

Michael Fu and C. Tantithamthavorn. 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), 2022. 374 citations.

0% Topic Match

Abstract: Software vulnerabilities are prevalent in software systems, causing a variety of problems including deadlock, information loss, or system failures. Thus, early predictions of software vulnerabilities are critically important in safety-critical software systems. Various ML/DL-based approaches have been proposed to predict vulnerabilities at the file/function/method level. Recently, IVDetect (a graph-based neural network) is proposed to predict vulnerabilities at the function level. Yet, the IVDetect approach is still inaccurate and coarse-grained. In this paper, we propose LINEVUL, a Transformer-based line-level vulnerability prediction approach in order to address several limitations of the state-of-the-art IVDetect approach. Through an empirical evaluation of a large-scale real-world...

[80] [ZARA: Improving Few-Shot Self-Rationalization for Small Language Models](#)

Wei-Lin Chen, ..., and Hsin-Hsi Chen. ArXiv, 2023. 11 citations.

0% Topic Match

Abstract: Language models (LMs) that jointly generate end-task answers as well as free-text rationales are known as self-rationalization models. Recent works demonstrate great performance gain for self-rationalization by few-shot prompting LMs with rationale-augmented exemplars. However, the ability to benefit from explanations only emerges with large-scale LMs, which have poor accessibility. In this work, we explore the less-studied setting of leveraging explanations for small LMs to improve few-shot self-rationalization. We first revisit the relationship between rationales and answers. Inspired by the implicit mental process of how human beings assess explanations, we present a novel approach, Zero-shot Augmentation of Rationale-Answer pairs (ZARA), to...

[81] [Generative transformations and patterns in LLM-native approaches for software verification and falsification](#)

V. Braberman, ..., and Rosiane de Freitas. ArXiv, 2024. 5 citations.

0% Topic Match

Abstract: The emergence of prompting as the dominant paradigm for leveraging Large Language Models (LLMs) has led to a proliferation of LLM-native software, where application behavior arises from complex, stochastic data transformations. However, the engineering of such systems remains largely exploratory and ad-hoc, hampered by the absence of conceptual frameworks, ex-ante methodologies, design guidelines, and specialized benchmarks. We argue that a foundational step

towards a more disciplined engineering practice is a systematic understanding of the core functional units--generative transformations--and their compositional patterns within LLM-native applications. Focusing on the rich domain of software verification and falsification, we conduct a secondary study of...

[82] [In-Context Vulnerability Propagation in LLMs \[Work In Progress Paper\]](#)

Aditya Nangia, ..., and Chinmay Kundu. Proceedings of the 30th ACM Symposium on Access Control Models and Technologies, 2025. 0 citations.
0% Topic Match

Abstract: The widespread adoption of Large Language Models (LLMs) in software development has accelerated coding workflows, with tools like GitHub Copilot and Google Gemini reducing development time by up to 55.8%. However, these systems suffer from security and trustworthiness challenges. In this paper, we demonstrate a novel attack - "Vulnerability propagation attack" in the context of the code generated by LLMs. We present a formal framework for evaluating vulnerability propagation in LLM-assisted development, focusing on two key metrics: Vulnerability Carry Rate (VCR), which quantifies the intra-session propagation of vulnerabilities, and Context Retention Factor (CRF), which measures cross-session persistence. Using three leading...

[83] [aiXcoder-7B-v2: Training LLMs to Fully Utilize the Long Context in Repository-level Code Completion](#)

Jia Li, ..., and Ge Li. ArXiv, 2025. 10 citations.

0% Topic Match

Abstract: Repository-level code completion aims to complete code based on the long contexts of the repository. Existing studies extract long contexts from the repository as inputs and leverage Large Language Models (LLMs) to generate code. However, we reveal a severe limitation of LLMs, i.e., LLMs may ignore the information within long contexts in code completion. In other words, even the contexts contain useful information (e.g., relevant APIs or similar code), LLMs may fail to utilize this information. We think this limitation is caused by an inherent bias in LLMs, i.e., relying on nearby contexts and ignoring long-range contexts. To address this,...

[84] ["Digital Camouflage": The LLVM Challenge in LLM-Based Malware Detection](#)

Ekin Böke and Simon Torka. ArXiv, 2025. 1 citations.

0% Topic Match

No summary or abstract available

[85] [Evaluation of ChatGPT Model for Vulnerability Detection](#)

A. Cheshkov, ..., and Rodion Levichev. ArXiv, 2023. 81 citations.

0% Topic Match

Abstract: In this technical report, we evaluated the performance of the ChatGPT and GPT-3 models for the task of vulnerability detection in code. Our evaluation was conducted on our real-world dataset, using binary and multi-label classification tasks on CWE vulnerabilities. We decided to evaluate the model because it has shown good performance on other code-based tasks, such as solving programming challenges and understanding code at a high level. However, we found that the ChatGPT model performed no better than a dummy classifier for both binary and multi-label classification tasks for code vulnerability detection.

[86] [Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection](#)

Niklas Risse and Marcel Bohme. USENIX Security Symposium, 2023. 42 citations.

0% Topic Match

Abstract: Recent results of machine learning for automatic vulnerability detection (ML4VD) have been very promising. Given only the source code of a function f , ML4VD techniques can decide if f contains a security flaw with up to 70% accuracy. However, as evident in our own experiments, the same top-performing models are unable to distinguish between functions that contain a vulnerability and functions where the vulnerability is patched. So, how can we explain this contradiction and how can we improve the way we evaluate ML4VD techniques to get a better picture of their actual capabilities? In this paper, we identify overfitting to...

[87] [Code Prompting Elicits Conditional Reasoning Abilities in Text+Code LLMs](#)

Haritz Puerto, ..., and Iryna Gurevych. ArXiv, 2024. 16 citations.

0% Topic Match

Abstract: Reasoning is a fundamental component of language understanding. Recent prompting techniques, such as chain of thought, have consistently improved LLMs' performance on various reasoning tasks. Nevertheless, there is still little understanding of what triggers reasoning abilities in LLMs in the inference stage. In this paper, we investigate the effect of the input representation on the reasoning abilities of LLMs. We hypothesize that representing natural language tasks as code can enhance specific reasoning abilities such as entity tracking or logical reasoning. To study this, we propose code prompting, a methodology we operationalize as a chain of prompts that transforms a natural...

[88] [Make a Feint to the East While Attacking in the West: Blinding LLM-Based Code Auditors with Flashboom Attacks](#)

Xiao Li, ..., and Fengyuan Xu. 2025 IEEE Symposium on Security and Privacy (SP), 2025. 1 citations.

0% Topic Match

Abstract: LLM-based vulnerability auditors (e.g., GitHub Copilot) represent a significant advancement in automated code analysis, offering precise detection of security vulnerabilities. This paper explores the potential to circumvent LLM-based vulnerability auditors by diverting their focus, decided by the LLM attention mechanism, away from real vulnerable code segments. In these LLM-based vulnerability auditors, the attention mechanism is supposed to focus on potentially vulnerable code sections to identify security issues. Our approach introduces high-attention code snippets (code fragments designed to draw focus) into the codebase under review. By strategically diverting the model's focus away from actual vulnerabilities, this technique effectively "blinds" the LLM,...

[89] [CodeUltraFeedback: An LLM-as-a-Judge Dataset for Aligning Large Language Models to Coding Preferences](#)

M. Weyssow, ..., and H. Sahraoui. ACM Transactions on Software Engineering and Methodology, 2024. 50 citations.

0% Topic Match

Abstract: Evaluating the alignment of large language models (LLMs) with user-defined coding preferences is a challenging endeavor that requires a deep assessment of LLMs' outputs. Existing methods and benchmarks rely primarily on automated metrics and static analysis tools, which often fail to capture the nuances of user instructions and LLM outputs. To address this gap, we introduce the LLM-as-a-Judge evaluation framework and present CodeUltraFeedback, a comprehensive dataset for assessing and improving LLM alignment with coding preferences. CodeUltraFeedback consists of 10,000 coding instructions, each annotated with four responses generated from a diverse pool of 14 LLMs. These responses are annotated using GPT-3.5...

[90] [ToolCoder: Teach Code Generation Models to use API search tools](#)

Kechi Zhang, ..., and Zhi Jin. ArXiv, 2023. 41 citations.

0% Topic Match

Abstract: Automatically generating source code from natural language descriptions has been a growing field of research in recent years. However, current large-scale code generation models often encounter difficulties when selecting appropriate APIs for specific contexts. These models may generate APIs that do not meet requirements or refer to non-existent APIs in third-party libraries, especially for lesser-known or private libraries. Inspired by the process of human developers using tools to search APIs, we propose ToolCoder, a novel approach that integrates API search tools with existing models to assist in code generation and API selection. To teach our model to use tools, we...

[91] [An empirical study of large language models for type and call graph analysis in Python and JavaScript](#)

Ashwini Venkatesh, ..., and Eric Bodden. Empirical Software Engineering, 2024. 0 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) are increasingly being explored for their potential in software engineering, particularly in static analysis tasks. In this study, we investigate the potential of current LLMs to enhance call-graph analysis and type inference for Python and JavaScript programs. We empirically evaluated 24 LLMs, including OpenAI's GPT series and open-source models like LLaMA and Mistral, using existing and newly developed benchmarks. Specifically, we enhanced TypeEvalPy, a micro-benchmarking framework for type inference in Python, with auto-generation capabilities, expanding its scope from 860 to 77,268 type annotations for Python. Additionally, we introduce SWARM-CG and SWARM-JS, comprehensive benchmarking suites for evaluating...

[92] [Detecting Semantic Clones of Unseen Functionality](#)

Konstantinos Kitsios, ..., and Alberto Bacchelli. ArXiv, 2025. 1 citations.

0% Topic Match

- Abstract: Semantic code clone detection is the task of detecting whether two snippets of code implement the same functionality (e.g., Sort Array). Recently, many neural models achieved near-perfect performance on this task. These models seek to make inferences based on their training data. Consequently, they better detect clones similar to those they have seen during training and may struggle to detect those they have not. Developers seeking clones are, of course, interested in both types of clones. We confirm this claim through a literature review, identifying three practical clone detection tasks in which the model's goal is to detect clones of...
- [93] [CONCORD: Clone-Aware Contrastive Learning for Source Code](#)
Yangruibo Ding, ..., and Baishakhi Ray. Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023. 11 citations.
0% Topic Match
Abstract: Deep Learning (DL) models to analyze source code have shown immense promise during the past few years. More recently, self-supervised pre-training has gained traction for learning generic code representations valuable for many downstream SE tasks, such as clone and bug detection. While previous work successfully learned from different code abstractions (e.g., token, AST, graph), we argue that it is also essential to factor in how developers code day-to-day for learning general-purpose representation. On the one hand, human developers tend to write repetitive programs referencing existing code snippets from the current codebase or online resources (e.g., Stack Overflow website) rather than...
- [94] [Catching Common Vulnerabilities with Code Language Models](#)
Syafiq Al Atiq, ..., and Kevin Dahlén. 2025 IEEE Secure Development Conference (SecDev), 2025. 0 citations.
0% Topic Match
Abstract: Code Language Model (code-LM)-based vulnerability detection for $\text{C} / \text{C}++$ faces a substantial challenge. Previous research has shown that even though it is better than any prior machine learning approach, it still struggles to generalize well, as shown by the low F1 score. Prior works treated the problem as a binary classification: either vulnerable or non-vulnerable. Looking deeper at the various vulnerability types, we see that this oversimplifies the problem, as different vulnerabilities have different characteristics. This paper investigates the same problem but with a different question, i.e., how would the model perform if the task is to classify whether...
- [95] [Human Ratings Do Not Reflect Downstream Utility: A Study of Free-Text Explanations for Model Predictions](#)
Jenny Kunz, ..., and Marco Kuhlmann. Unknown journal, 2022. 8 citations.
0% Topic Match
Abstract: Models able to generate free-text rationales that explain their output have been proposed as an important step towards interpretable NLP for "reasoning" tasks such as natural language inference and commonsense question answering. However, the relative merits of different architectures and types of rationales are not well understood and hard to measure. In this paper, we contribute two insights to this line of research: First, we find that models trained on gold explanations learn to rely on these but, in the case of the more challenging question answering data set we use, fail when given generated explanations at test time. However,...
- [96] [JudgeBench: A Benchmark for Evaluating LLM-based Judges](#)
Sijun Tan, ..., and Ion Stoica. ArXiv, 2024. 123 citations.
0% Topic Match
Abstract: LLM-based judges have emerged as a scalable alternative to human evaluation and are increasingly used to assess, compare, and improve models. However, the reliability of LLM-based judges themselves is rarely scrutinized. As LLMs become more advanced, their responses grow more sophisticated, requiring stronger judges to evaluate them. Existing benchmarks primarily focus on a judge's alignment with human preferences, but often fail to account for more challenging tasks where crowdsourced human preference is a poor indicator of factual and logical correctness. To address this, we propose a novel evaluation framework to objectively evaluate LLM-based judges. Based on this framework, we propose...
- [97] [Learning Code Preference via Synthetic Evolution](#)
Jiawei Liu, ..., and Zijian Wang. ArXiv, 2024. 16 citations.
0% Topic Match
Abstract: Large Language Models (LLMs) have recently demonstrated remarkable coding capabilities. However, assessing code generation based on well-formed properties and aligning it with developer preferences remains challenging. In this paper, we explore two key questions under the new challenge of code preference learning: (i) How do we train models to predict meaningful preferences for code? and (ii) How do human and LLM preferences align with verifiable code properties and developer code tastes? To this end, we propose CodeFavor, a framework for training pairwise code preference models from synthetic evolution data, including code commits and code critiques. To evaluate code preferences, we...
- [98] [Repair Is Nearly Generation: Multilingual Program Repair with LLMs](#)
Harshit Joshi, ..., and Gust Verbruggen. Unknown journal, 2022. 163 citations.
0% Topic Match
Abstract: Most programmers make mistakes when writing code. Some of these mistakes are small and require few edits to the original program – a class of errors recently termed last mile mistakes. These errors break the flow for experienced developers and can stump novice programmers. Existing automated repair techniques targeting this class of errors are language-specific and do not easily carry over to new languages. Transferring symbolic approaches requires substantial engineering and neural approaches require data and retraining. We introduce RING, a multilingual repair engine powered by a large language model trained on code (LLMC) such as Codex. Such a multilingual...
- [99] [AutoSDT: Scaling Data-Driven Discovery Tasks Toward Open Co-Scientists](#)
Yifei Li, ..., and Huan Sun. ArXiv, 2025. 2 citations.
0% Topic Match
Abstract: Despite long-standing efforts in accelerating scientific discovery with AI, building AI co-scientists remains challenging due to limited high-quality data for training and evaluation. To tackle this data scarcity issue, we present AutoSDT, an automatic pipeline that collects high-quality coding tasks in real-world data-driven discovery workflows. AutoSDT leverages the coding capabilities and parametric knowledge of LLMs to search for diverse sources, select ecologically valid tasks, and synthesize accurate task instructions and code solutions. Using our pipeline, we construct AutoSDT-5K, a dataset of 5,404 coding tasks for data-driven discovery that covers four scientific disciplines and 756 unique Python packages. To the best...
- [100] [LoRA: Low-Rank Adaptation of Large Language Models](#)
J. E. Hu, ..., and Weizhu Chen. ArXiv, 2021. 14160 citations.
0% Topic Match
Abstract: An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which re-trains all model parameters, becomes less feasible. Using GPT-3 175B as an example -- deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to GPT-3 175B fine-tuned with Adam, LoRA...
- [101] [PVDetector: Pretrained Vulnerability Detection on Vulnerability-enriched Code Semantic Graph](#)
Jiayuan Li, ..., and Hongsong Zhu. ACM Transactions on Software Engineering and Methodology, 2025. 0 citations.
0% Topic Match
Abstract: Automated vulnerability detection is a critical issue in software security. The advent of deep learning (DL) has led to numerous studies employing DL to detect vulnerabilities in software source code. However, existing approaches still perform poorly, particularly with real-world vulnerabilities, due to the difficulty in accurately capturing their properties. To this end, we introduce PVDetector, a DL-based approach that utilizes rich code semantics, incorporates vulnerability knowledge, and leverages pretrained code representations for precise vulnerability detection. At its core, PVDetector employs a new model called Vulnerability-enriched Code Semantic Graph (VCSG), which accurately characterizes functions by distinguishing the semantics of identical variables...
- [102] [Auto-Evolve: Enhancing Large Language Model's Performance via Self-Reasoning Framework](#)
Krishna Aswani, ..., and Simon Lacasse. Unknown journal, 2024. 1 citations.
0% Topic Match
Abstract: Recent advancements in prompt engineering strategies, such as Chain-of-Thought (CoT) and Self-Discover, have demonstrated significant potential in improving the reasoning abilities of Large Language Models (LLMs). However, these state-of-the-art (SOTA) prompting strategies rely on single or fixed set of static seed reasoning modules like "think step by step" or "break down this problem" intended to simulate human approach to problem-solving. This constraint limits the

flexibility of models in tackling diverse problems effectively. In this paper, we introduce Auto-Evolve, a novel framework that enables LLMs to self-create dynamic reasoning modules and downstream action plan, resulting in significant improvements over current SOTA methods. We evaluate Auto-Evolve...

[103] [What if you said that differently?: How Explanation Formats Affect Human Feedback Efficacy and User Perception](#)

Chaitanya Malaviya, ..., and Mark Yatskar. Unknown journal, 2023. 2 citations.

0% Topic Match

Abstract: Eliciting feedback from end users of NLP models can be beneficial for improving models. However, how should we present model responses to users so they are most amenable to be corrected from user feedback? Further, what properties do users value to understand and trust responses? We answer these questions by analyzing the effect of rationales (or explanations) generated by QA models to support their answers. We specifically consider decomposed QA models that first extract an intermediate rationale based on a context and a question and then use solely this rationale to answer the question. A rationale outlines the approach followed...

[104] [SelfCodeAlign: Self-Alignment for Code Generation](#)

Yuxiang Wei, ..., and Lingming Zhang. ArXiv, 2024. 48 citations.

0% Topic Match

Abstract: Instruction tuning is a supervised fine-tuning approach that significantly improves the ability of large language models (LLMs) to follow human instructions. We propose SelfCodeAlign, the first fully transparent and permissive pipeline for self-aligning code LLMs without extensive human annotations or distillation. SelfCodeAlign employs the same base model for inference throughout the data generation process. It first extracts diverse coding concepts from high-quality seed snippets to generate new tasks. It then samples multiple responses per task, pairs each with test cases, and validates them in a sandbox environment. Finally, passing examples are selected for instruction tuning. In our primary experiments, we...

[105] [Automatic Prompt Generation Based on Combinatorial Relationships in Reasoning Problems](#)

Tong Wu and Hao He. 2024 6th International Conference on Natural Language Processing (ICNLP), 2024. 1 citations.

0% Topic Match

Abstract: The effectiveness of Large Language Models (LLMs) in tasks involving reasoning is significantly influenced by the structure and formulation of the prompts, contemporary research in prompt engineering aims to help LLMs better understand the paradigms of reasoning question (e.g., CoT). However, these efforts have either struggled to effectively incorporate external knowledge into single prompt or integrating entire corpus information, often fails to significantly enhance the reasoning capabilities of LLMs. This paper introduces a novel prompting method that incorporates implicit hints that represent logical combinatorial relationships between known conditions in reasoning problems, guiding LLMs to think correctly in the initial steps...

[106] [Algorithmic Inversion: A Learnable Algorithm Representation for Code Generation](#)

Zhongyi Shi, ..., and Yanjun Wu. 2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC), 2025. 0 citations.

0% Topic Match

Abstract: The prevalent fine-tuning paradigm for large language models (LLMs) has demonstrated strong performance on various code generation tasks. However, these models still fall short when confronted with algorithmic programming problems, where precise algorithmic reasoning is required. Humans typically adopt diverse algorithmic techniques to tackle complex programming problems, enabling general analysis and accurate implementation. Building on this observation, we propose a method that learns compact, LLM-friendly representations of algorithmic knowledge, termed Algorithmic Inversion (AI), which aims to aid LLMs in understanding programming problems. Specifically, we apply a lightweight fine-tuning process on codeoriented models to automatically learn algorithm embeddings. When concatenated with...

[107] [CodeBERT: A Pre-Trained Model for Programming and Natural Languages](#)

Zhangyin Feng, ..., and Ming Zhou. ArXiv, 2020. 3206 citations.

0% Topic Match

Abstract: We present CodeBERT, a bimodal pre-trained model for programming language (PL) and natural language (NL). CodeBERT learns general-purpose representations that support downstream NL-PL applications such as natural language code search, code documentation generation, etc. We develop CodeBERT with Transformer-based neural architecture, and train it with a hybrid objective function that incorporates the pre-training task of replaced token detection, which is to detect plausible alternatives sampled from generators. This enables us to utilize both "bimodal" data of NL-PL pairs and "unimodal" data, where the former provides input tokens for model training while the latter helps to learn better generators. We evaluate...

[108] [A Survey on Code Generation with LLM-based Agents](#)

Yihong Dong, ..., and Ge Li. ArXiv, 2025. 19 citations.

0% Topic Match

Abstract: Code generation agents powered by large language models (LLMs) are revolutionizing the software development paradigm. Distinct from previous code generation techniques, code generation agents are characterized by three core features. 1) Autonomy: the ability to independently manage the entire workflow, from task decomposition to coding and debugging. 2) Expanded task scope: capabilities that extend beyond generating code snippets to encompass the full software development lifecycle (SDLC). 3) Enhancement of engineering practicality: a shift in research emphasis from algorithmic innovation toward practical engineering challenges, such as system reliability, process management, and tool integration. This domain has recently witnessed rapid development and...

[109] [ReAct: Synergizing Reasoning and Acting in Language Models](#)

Shunyu Yao, ..., and Yuan Cao. ArXiv, 2022. 4570 citations.

0% Topic Match

Abstract: While large language models (LLMs) have demonstrated impressive capabilities across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with external sources, such as knowledge bases or environments, to gather additional...

[110] [Explanation Selection Using Unlabeled Data for Chain-of-Thought Prompting](#)

Xi Ye and Greg Durrett. Unknown journal, 2023. 14 citations.

0% Topic Match

Abstract: Recent work has shown how to prompt large language models with explanations to obtain strong performance on textual reasoning tasks, i.e., the chain-of-thought paradigm. However, subtly different explanations can yield widely varying downstream task accuracy. Explanations that have not been "tuned" for a task, such as off-the-shelf explanations written by nonexperts, may lead to mediocre performance. This paper tackles the problem of how to optimize explanation-infused prompts in a blackbox fashion. We first generate sets of candidate explanations for each example in the prompt using a leave-one-out scheme, then find an effective combination of these explanations with a two-stage framework. We first...

[111] [MALPRE: Malware Protocol Reverse Engineering through Code Slicing and Agentic Workflow](#)

Yuyao Huang, ..., and Guoyu Huo. 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE), 2025. 0 citations.

0% Topic Match

Abstract: Clarifying malware communication protocols is critical for enhancing system security. Existing protocol reverse engineering (PRE) methods lack effective strategies, failing to recover protocol structures or infer precise field semantics. To address these challenges, we propose MALPRE, an execution trace-based PRE framework that integrates precise program analysis with large language models (LLMs) for automated malware protocol format recovery. MALPRE first embeds and hierarchically clusters the code slices to restore message formats. It then introduces a multi-agentic workflow comprising code analyst, malware expert, and protocol puzzler roles to collaboratively infer field semantics. Evaluations on the dataset containing six popular malware frameworks demonstrate...

[112] [Think Beyond Size: Adaptive Prompting for More Effective Reasoning](#)

R. Kamesh. ArXiv, 2024. 2 citations.

0% Topic Match

Abstract: Pretrained large language models (LLMs) are increasingly utilized across a wide range of natural language processing (NLP) tasks due to their impressive capabilities as few-shot learners. Recent techniques, such as chain-of-thought (CoT) prompting, have significantly advanced multi-step reasoning by introducing step-by-step decomposition, achieving state-of-the-art results on complex reasoning benchmarks. However, these approaches often rely on static prompting templates that do not adapt to task complexity or errors during the reasoning process. In this work, we introduce Adaptive Prompting, a dynamic and iterative

framework designed to enhance reasoning by incorporating real-time adjustments to prompt structures and validation mechanisms. Experimental results demonstrate that...

[113] [From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge](#)

Dawei Li, ..., and Huan Liu. ArXiv, 2024. 213 citations.

0% Topic Match

Abstract: Assessment and evaluation have long been critical challenges in artificial intelligence (AI) and natural language processing (NLP). Traditional methods, usually matching-based or small model-based, often fall short in open-ended and dynamic scenarios. Recent advancements in Large Language Models (LLMs) inspire the "LLM-as-a-judge" paradigm, where LLMs are leveraged to perform scoring, ranking, or selection for various machine learning evaluation scenarios. This paper presents a comprehensive survey of LLM-based judgment and assessment, offering an in-depth overview to review this evolving field. We first provide the definition from both input and output perspectives. Then we introduce a systematic taxonomy to explore LLM-as-a-judge along three dimensions:...

[114] [Diagnostics-Guided Explanation Generation](#)

Pepa Atanasova, ..., and Isabelle Augenstein. Unknown journal, 2021. 9 citations.

0% Topic Match

Abstract: Explanations shed light on a machine learning model's rationales and can aid in identifying deficiencies in its reasoning process. Explanation generation models are typically trained in a supervised way given human explanations. When such annotations are not available, explanations are often selected as those portions of the input that maximise a downstream task's performance, which corresponds to optimising an explanation's Faithfulness to a given model. Faithfulness is one of several so-called diagnostic properties, which prior work has identified as useful for gauging the quality of an explanation without requiring annotations. Other diagnostic properties are Data Consistency, which measures how similar...

[115] [ChatCoder: Chat-based Refine Requirement Improves LLMs' Code Generation](#)

Zejun Wang, ..., and Zhi Jin. ArXiv, 2023. 12 citations.

0% Topic Match

Abstract: Large language models have shown good performances in generating code to meet human requirements. However, human requirements expressed in natural languages can be vague, incomplete, and ambiguous, leading large language models to misunderstand human requirements and make mistakes. Worse, it is difficult for a human user to refine the requirement. To help human users refine their requirements and improve large language models' code generation performances, we propose ChatCoder: a method to refine the requirements via chatting with large language models. We design a chat scheme in which the large language models will guide the human users to refine their expression...

[116] [Large Language Models are Zero-Shot Reasoners](#)

Takeshi Kojima, ..., and Yusuke Iwasawa. ArXiv, 2022. 5715 citations.

0% Topic Match

Abstract: Pretrained large language models (LLMs) are widely used in many sub-fields of natural language processing (NLP) and generally known as excellent few-shot learners with task-specific exemplars. Notably, chain of thought (CoT) prompting, a recent technique for eliciting complex multi-step reasoning through step-by-step answer examples, achieved the state-of-the-art performances in arithmetics and symbolic reasoning, difficult system-2 tasks that do not follow the standard scaling laws for LLMs. While these successes are often attributed to LLMs' ability for few-shot learning, we show that LLMs are decent zero-shot reasoners by simply adding "Let's think step by step" before each answer. Experimental results demonstrate that our...

[117] [Prompting for explanations improves Adversarial NLI. Is this true? {Yes} it is {true} because {it weakens superficial cues}](#)

Pride Kavumba, ..., and Kentaro Inui. Unknown journal, 2023. 14 citations.

0% Topic Match

Abstract: Explanation prompts ask language models to not only assign a particular label to a given input, such as true, entailment, or contradiction in the case of natural language inference but also to generate a free-text explanation that supports this label. For example: "This is {label} because {explanation}." While this type of prompt was originally introduced with the aim of improving model interpretability, we show where that explanation prompts also improve robustness to adversarial perturbations in natural language inference benchmarks. Compared to prompting for labels only, explanation prompting consistently yields stronger performance on adversarial benchmarks, outperforming the state of the art on Adversarial Natural...

[118] [A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models](#)

Wenqi Fan, ..., and Qing Li. Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024. 512 citations.

0% Topic Match

Abstract: As one of the most advanced techniques in AI, Retrieval-Augmented Generation (RAG) can offer reliable and up-to-date external knowledge, providing huge convenience for numerous tasks. Particularly in the era of AI-Generated Content (AIGC), the powerful capacity of retrieval in providing additional knowledge enables RAG to assist existing generative AI in producing high-quality outputs. Recently, Large Language Models (LLMs) have demonstrated revolutionary abilities in language understanding and generation, while still facing inherent limitations such as hallucinations and out-of-date internal knowledge. Given the powerful abilities of RAG in providing the latest and helpful auxiliary information, Retrieval-Augmented Large Language Models (RA-LLMs) have emerged...

[119] [CVEfixes: automated collection of vulnerabilities and their fixes from open-source software](#)

G. Bhandari, ..., and L. Moonen. Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering, 2021. 212 citations.

0% Topic Match

Abstract: Data-driven research on the automated discovery and repair of security vulnerabilities in source code requires comprehensive datasets of real-life vulnerable code and their fixes. To assist in such research, we propose a method to automatically collect and curate a comprehensive vulnerability dataset from Common Vulnerabilities and Exposures (CVE) records in the National Vulnerability Database (NVD). We implement our approach in a fully automated dataset collection tool and share an initial release of the resulting vulnerability dataset named CVEfixes. The CVEfixes collection tool automatically fetches all available CVE records from the NVD, gathers the vulnerable code and corresponding fixes from associated...

[120] [PSM: Prompt Sensitivity Minimization via LLM-Guided Black-Box Optimization](#)

Huseein Jawad and N. Brunel. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: System prompts are critical for guiding the behavior of Large Language Models (LLMs), yet they often contain proprietary logic or sensitive information, making them a prime target for extraction attacks. Adversarial queries can successfully elicit these hidden instructions, posing significant security and privacy risks. Existing defense mechanisms frequently rely on heuristics, incur substantial computational overhead, or are inapplicable to models accessed via black-box APIs. This paper introduces a novel framework for hardening system prompts through shield appending, a lightweight approach that adds a protective textual layer to the original prompt. Our core contribution is the formalization of prompt hardening as...

[121] [Testing and Understanding Erroneous Planning in LLM Agents through Synthesized User Inputs](#)

Zhenlan Ji, ..., and Shuai Wang. ArXiv, 2024. 15 citations.

0% Topic Match

Abstract: Agents based on large language models (LLMs) have demonstrated effectiveness in solving a wide range of tasks by integrating LLMs with key modules such as planning, memory, and tool usage. Increasingly, customers are adopting LLM agents across a variety of commercial applications critical to reliability, including support for mental well-being, chemical synthesis, and software development. Nevertheless, our observations and daily use of LLM agents indicate that they are prone to making erroneous plans, especially when the tasks are complex and require long-term planning. In this paper, we propose PDoctor, a novel and automated approach to testing LLM agents and understanding...

[122] [Commonsense Knowledge Reasoning and Generation with Pre-trained Language Models: A Survey](#)

Prajwal Bhargava and Vincent Ng. Unknown journal, 2022. 73 citations.

0% Topic Match

Abstract: While commonsense knowledge acquisition and reasoning has traditionally been a core research topic in the knowledge representation and reasoning community, recent years have seen a surge of interest in the natural language processing community in developing pre-trained models and testing their ability to address a variety of newly designed commonsense knowledge reasoning and generation tasks. This paper presents a survey of these tasks, discusses the strengths

and weaknesses of state-of-the-art pre-trained models for commonsense reasoning and generation as revealed by these tasks, and reflects on future research directions.

[123] [VulDeeLocator: A Deep Learning-Based Fine-Grained Vulnerability Detector](#)

Zhuguo Li, ..., and Hai Jin. IEEE Transactions on Dependable and Secure Computing, 2020. 224 citations.

0% Topic Match

Abstract: Automatically detecting software vulnerabilities is an important problem that has attracted much attention from the academic research community. However, existing vulnerability detectors still cannot achieve the vulnerability detection capability and the locating precision that would warrant their adoption for real-world use. In this article, we present a vulnerability detector that can simultaneously achieve a high detection capability and a high locating precision, dubbed Vulnerability Deep learning-based Locator (VulDeeLocator). In the course of designing VulDeeLocator, we encounter difficulties including how to accommodate semantic relations between the definitions of types as well as macros and their uses across files, how to accommodate...

[124] [ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery](#)

Ziru Chen, ..., and Huan Sun. International Conference on Learning Representations, 2025. 29 citations.

0% Topic Match

No summary or abstract available

[125] [Rationale-Augmented Ensembles in Language Models](#)

Xuezhi Wang, ..., and Denny Zhou. ArXiv, 2022. 135 citations.

0% Topic Match

Abstract: Recent research has shown that rationales, or step-by-step chains of thought, can be used to improve performance in multi-step reasoning tasks. We reconsider rationale-augmented prompting for few-shot in-context learning, where (input ->output) prompts are expanded to (input, rationale ->output) prompts. For rationale-augmented prompting we demonstrate how existing approaches, which rely on manual prompt engineering, are subject to sub-optimal rationales that may harm performance. To mitigate this brittleness, we propose a unified framework of rationale-augmented ensembles, where we identify rationale sampling in the output space as the key component to robustly improve performance. This framework is general and can easily be...

[126] [Zephyr: Direct Distillation of LM Alignment](#)

Lewis Tunstall, ..., and Thomas Wolf. ArXiv, 2023. 494 citations.

0% Topic Match

Abstract: We aim to produce a smaller language model that is aligned to user intent. Previous research has shown that applying distilled supervised fine-tuning (dSFT) on larger models significantly improves task accuracy; however, these models are unaligned, i.e. they do not respond well to natural prompts. To distill this property, we experiment with the use of preference data from AI Feedback (AIF). Starting from a dataset of outputs ranked by a teacher model, we apply distilled direct preference optimization (dDPO) to learn a chat model with significantly improved intent alignment. The approach requires only a few hours of training without any...

[127] [Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback](#)

Yuntao Bai, ..., and Jared Kaplan. ArXiv, 2022. 3295 citations.

0% Topic Match

Abstract: We apply preference modeling and reinforcement learning from human feedback (RLHF) to tune language models to act as helpful and harmless assistants. We find this alignment training improves performance on almost all NLP evaluations, and is fully compatible with training for specialized skills such as python coding and summarization. We explore an iterated online mode of training, where preference models and RL policies are updated on a weekly cadence with fresh human feedback data, efficiently improving our datasets and models. Finally, we investigate the robustness of RLHF training, and identify a roughly linear relation between the RL reward and the...

[128] [Measuring Mathematical Problem Solving With the MATH Dataset](#)

Dan Hendrycks, ..., and J. Steinhardt. ArXiv, 2021. 3584 citations.

0% Topic Match

Abstract: Many intellectual endeavors require mathematical problem solving, but this skill remains beyond the capabilities of computers. To measure this ability in machine learning models, we introduce MATH, a new dataset of 12,500 challenging competition mathematics problems. Each problem in MATH has a full step-by-step solution which can be used to teach models to generate answer derivations and explanations. To facilitate future research and increase accuracy on MATH, we also contribute a large auxiliary pretraining dataset which helps teach models the fundamentals of mathematics. Even though we are able to increase accuracy on MATH, our results show that accuracy remains relatively...

[129] [CARE: Turning LLMs Into Causal Reasoning Expert](#)

Juncheng Dong, ..., and David E. Carlson. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large language models (LLMs) have recently demonstrated impressive capabilities across a range of reasoning and generation tasks. However, research studies have shown that LLMs lack the ability to identify causal relationships, a fundamental cornerstone of human intelligence. We first conduct an exploratory investigation of LLMs' behavior when asked to perform a causal-discovery task and find that they mostly rely on the semantic meaning of variable names, ignoring the observation data. This is unsurprising, given that LLMs were never trained to process structural datasets. To first tackle this challenge, we prompt the LLMs with the outputs of established causal discovery algorithms designed...

[130] [BERTScore: Evaluating Text Generation with BERT](#)

Tianyi Zhang, ..., and Yoav Artzi. ArXiv, 2019. 7035 citations.

0% Topic Match

Abstract: We propose BERTScore, an automatic evaluation metric for text generation. Analogously to common metrics, BERTScore computes a similarity score for each token in the candidate sentence with each token in the reference sentence. However, instead of exact matches, we compute token similarity using contextual embeddings. We evaluate using the outputs of 363 machine translation and image captioning systems. BERTScore correlates better with human judgments and provides stronger model selection performance than existing metrics. Finally, we use an adversarial paraphrase detection task to show that BERTScore is more robust to challenging examples when compared to existing metrics.

[131] [BixBench: a Comprehensive Benchmark for LLM-based Agents in Computational Biology](#)

Ludovico Mitchener, ..., and Samuel G. Rodrigues. ArXiv, 2025. 27 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) and LLM-based agents show great promise in accelerating scientific research. Existing benchmarks for measuring this potential and guiding future development continue to evolve from pure recall and rote knowledge tasks, towards more practical work such as literature review and experimental planning. Bioinformatics is a domain where fully autonomous AI-driven discovery may be near, but no extensive benchmarks for measuring progress have been introduced to date. We therefore present the Bioinformatics Benchmark (BixBench), a dataset comprising over 50 real-world scenarios of practical biological data analysis with nearly 300 associated open-answer questions designed to measure the ability of...

[132] [Beyond LLMs: An Exploration of Small Open-source Language Models in Logging Statement Generation](#)

Renyi Zhong and M. R. Lyu. Journal Not Provided, Unknown year. 1 citations.

0% Topic Match

No summary or abstract available

[133] [Towards LLM-Assisted Vulnerability Detection and Repair for Open-Source 5G UE Implementations](#)

Rupam Patir, ..., and Hongxin Hu. Journal Not Provided, Unknown year. 1 citations.

0% Topic Match

No summary or abstract available

[134] [Beyond Prompt Engineering: The Evolution of Reasoning in Advanced Large Language Models](#)

Nan Wu. International Journal on Science and Technology, 2025. 0 citations.

0% Topic Match

Abstract: This paper explores the evolving role of prompt engineering as large language models (LLMs) develop enhanced intrinsic reasoning capabilities. Initially essential for effective model performance, explicit prompting techniques are becoming less crucial with advanced models like GPT-4.5 and DeepSeek R1. Benchmark analyses indicate that intrinsic reasoning now solves most reasoning tasks efficiently, though explicit prompting still provides incremental benefits in specialized scenarios. Future directions emphasize intrinsic reasoning improvements, automated prompting strategies, and refined evaluation methods, marking a fundamental shift in leveraging LLMs.

[135] [Open Problems and Fundamental Limitations of Reinforcement Learning from Human Feedback](#)

Stephen Casper, ..., and Dylan Hadfield-Menell. ArXiv, 2023. 669 citations.

0% Topic Match

Abstract: Reinforcement learning from human feedback (RLHF) is a technique for training AI systems to align with human goals. RLHF has emerged as the central method used to finetune state-of-the-art large language models (LLMs). Despite this popularity, there has been relatively little public work systematizing its flaws. In this paper, we (1) survey open problems and fundamental limitations of RLHF and related methods; (2) overview techniques to understand, improve, and complement RLHF in practice; and (3) propose auditing and disclosure standards to improve societal oversight of RLHF systems. Our work emphasizes the limitations of RLHF and highlights the importance of a...

[136] [Why Can GPT Learn In-Context? Language Models Implicitly Perform Gradient Descent as Meta-Optimizers](#)

Damai Dai, ..., and Furu Wei. ArXiv, 2022. 205 citations.

0% Topic Match

Abstract: Large pretrained language models have shown surprising in-context learning (ICL) ability. With a few demonstration input-label pairs, they can predict the label for an unseen input without parameter updates. Despite the great success in performance, its working mechanism still remains an open question. In this paper, we explain language models as meta-optimizers and understand in-context learning as implicit finetuning. Theoretically, we figure out that Transformer attention has a dual form of gradient descent. On top of it, we understand ICL as follows: GPT first produces meta-gradients according to the demonstration examples, and then these meta-gradients are applied to the original...

[137] [Bleu: a Method for Automatic Evaluation of Machine Translation](#)

Kishore Papineni, ..., and Wei-Jing Zhu. Unknown journal, 2002. 30504 citations.

0% Topic Match

Abstract: Human evaluations of machine translation are extensive but expensive. Human evaluations can take months to finish and involve human labor that can not be reused. We propose a method of automatic machine translation evaluation that is quick, inexpensive, and language-independent, that correlates highly with human evaluation, and that has little marginal cost per run. We present this method as an automated understudy to skilled human judges which substitutes for them when there is need for quick or frequent evaluations.

[138] [A Survey on Context Learning](#)

Guangxu Xun, ..., and Aidong Zhang. IEEE Transactions on Knowledge and Data Engineering, 2017. 1299 citations.

0% Topic Match

No summary or abstract available

[139] [Formalization of text prompts to artificial intelligence systems](#)

Vladyslav Oliinyk, ..., and Victor Chelombitko. Eastern-European Journal of Enterprise Technologies, 2025. 0 citations.

0% Topic Match

Abstract: This study's object is the process that formalizes text prompts to large language models of artificial intelligence for the purpose of automatically generating action cards for hexagonal tabletop wargames. The task relates to the ambiguity (42% of terms are misinterpreted), contextual incompleteness (37%), and syntactic variability (21%) of natural language prompts, resulting in unhelpful and unpredictable responses. To address this problem, a conceptual-practical model has been proposed, which combines structured prompt templates, a localized glossary of key terms, as well as clear instructions on response format. Practical verification was carried out by generating a set of action cards for solo...

[140] [Dual Prompt-Based Few-Shot Learning for Automated Vulnerability Patch Localization](#)

Junwei Zhang, ..., and Shanping Li. 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2024. 6 citations.

0% Topic Match

Abstract: Vulnerabilities are disclosed with corresponding patches so that users can remediate them in time. However, there are instances where patches are not released with the disclosed vulnerabilities, causing hidden dangers, especially if dependent software remains uninformed about the affected code repository. Hence, it is crucial to automatically locate security patches for disclosed vulnerabilities among a multitude of commits. Despite the promising performance of existing learning-based localization approaches, they still suffer from the following limitations: (1) They cannot perform well in data scarcity scenarios. Most neural models require extensive datasets to capture the semantic correlations between the vulnerability description and code...

[141] [VulnLLMEval: A Framework for Evaluating Large Language Models in Software Vulnerability Detection and Patching](#)

Arastoo Zibaeirad and Marco Vieira. ArXiv, 2024. 13 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have shown promise in tasks like code translation, prompting interest in their potential for automating software vulnerability detection (SVD) and patching (SVP). To further research in this area, establishing a benchmark is essential for evaluating the strengths and limitations of LLMs in these tasks. Despite their capabilities, questions remain regarding whether LLMs can accurately analyze complex vulnerabilities and generate appropriate patches. This paper introduces VulnLLMEval, a framework designed to assess the performance of LLMs in identifying and patching vulnerabilities in C code. Our study includes 307 real-world vulnerabilities extracted from the Linux kernel, creating a well-curated...

[142] [Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories](#)

Alperen Yildiz, ..., and Dinil Mon Divakaran. ArXiv, 2025. 11 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have shown promise in software vulnerability detection, particularly on function-level benchmarks like Devign and BigVul. However, real-world detection requires interprocedural analysis, as vulnerabilities often emerge through multi-hop function calls rather than isolated functions. While repository-level benchmarks like ReposVul and VulEval introduce interprocedural context, they remain computationally expensive, lack pairwise evaluation of vulnerability fixes, and explore limited context retrieval, limiting their practicality. We introduce JitVul, a JIT vulnerability detection benchmark linking each function to its vulnerability-introducing and fixing commits. Built from 879 CVEs spanning 91 vulnerability types, JitVul enables comprehensive evaluation of detection capabilities. Our results show...

[143] [Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks](#)

Yaqin Zhou, ..., and Yang Liu. ArXiv, 2019. 927 citations.

Not measured Topic Match

Abstract: Vulnerability identification is crucial to protect the software systems from attacks for cyber security. It is especially important to localize the vulnerable functions among the source code to facilitate the fix. However, it is a challenging and tedious process, and also requires specialized security expertise. Inspired by the work on manually-defined patterns of vulnerabilities from various code representation graphs and the recent advance on graph neural networks, we propose Devign, a general graph neural network based model for graph-level classification through learning on a rich set of code semantic representations. It includes a novel Conv module to efficiently extract useful...

[144] [VulDetectBench: Evaluating the Deep Capability of Vulnerability Detection with Large Language Models](#)

Yu Liu, ..., and Wei Chen. ArXiv, 2024. 12 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have training corpora containing large amounts of program code, greatly improving the model's code comprehension and generation capabilities. However, sound comprehensive research on detecting program vulnerabilities, a more specific task related to code, and evaluating

the performance of LLMs in this more specialized scenario is still lacking. To address common challenges in vulnerability analysis, our study introduces a new benchmark, VulDetectBench, specifically designed to assess the vulnerability detection capabilities of LLMs. The benchmark comprehensively evaluates LLM's ability to identify, classify, and locate vulnerabilities through five tasks of increasing difficulty. We evaluate the performance of 17 models...

[145] [Language Models are Few-Shot Learners](#)

Tom B. Brown, ..., and Dario Amodei. ArXiv, 2020. 50021 citations.

Not measured Topic Match

Abstract: Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions - something which current NLP systems still largely struggle to do. Here we show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically,...

[146] [Teaching Large Language Models to Reason with Reinforcement Learning](#)

Alex Havrilla, ..., and R. Raileanu. ArXiv, 2024. 135 citations.

Not measured Topic Match

Abstract: Reinforcement Learning from Human Feedback (RLHF) has emerged as a dominant approach for aligning LLM outputs with human preferences. Inspired by the success of RLHF, we study the performance of multiple algorithms that learn from feedback (Expert Iteration, Proximal Policy Optimization (PPO)), Return-Conditioned RL) on improving LLM reasoning capabilities. We investigate both sparse and dense rewards provided to the LLM both heuristically and via a learned reward model. We additionally start from multiple model sizes and initializations both with and without supervised fine-tuning (SFT) data. Overall, we find all algorithms perform comparably, with Expert Iteration performing best in most cases....

[147] [Your Instructions Are Not Always Helpful: Assessing the Efficacy of Instruction Fine-tuning for Software Vulnerability Detection](#)

Imam Nur Bani Yusuf and Lingxiao Jiang. ArXiv, 2024. 7 citations.

Not measured Topic Match

Abstract: Software, while beneficial, poses potential cybersecurity risks due to inherent vulnerabilities. Detecting these vulnerabilities is crucial, and deep learning has shown promise as an effective tool for this task due to its ability to perform well without extensive feature engineering. However, a challenge in deploying deep learning for vulnerability detection is the limited availability of training data. Recent research highlights the deep learning efficacy in diverse tasks. This success is attributed to instruction fine-tuning, a technique that remains under-explored in the context of vulnerability detection. This paper investigates the capability of models, specifically a recent language model, to generalize beyond...

[148] [When LLMs meet cybersecurity: a systematic literature review](#)

Jie Zhang, ..., and Dan Meng. Cybersecurity, 2024. 120 citations.

Not measured Topic Match

Abstract: The rapid development of large language models (LLMs) has opened new avenues across various fields, including cybersecurity, which faces an evolving threat landscape and demand for innovative technologies. Despite initial explorations into the application of LLMs in cybersecurity, there is a lack of a comprehensive overview of this research area. This paper addresses this gap by providing a systematic literature review, covering the analysis of over 300 works, encompassing 25 LLMs and more than 10 downstream scenarios. Our comprehensive overview addresses three key research questions: the construction of cybersecurity-oriented LLMs, the application of LLMs to various cybersecurity tasks, the challenges...

[149] [From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security](#)

Enna BaSi and A. Giaretta ArXiv, 2024. 7 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have emerged as powerful tools for automating various programming tasks, including security-related ones, such as detecting and fixing vulnerabilities. Despite their promising capabilities, when required to produce or modify pre-existing code, LLMs could introduce vulnerabilities unknown to the programmer. When analyzing code, they could miss clear vulnerabilities or signal nonexistent ones. In this Systematic Literature Review (SLR), we aim to investigate both the security benefits and potential drawbacks of using LLMs for a variety of code-related tasks. In particular, first we focus on the types of vulnerabilities that could be introduced by LLMs, when used for...

[150] [Revisiting Pre-trained Language Models for Vulnerability Detection](#)

Youpeng Li, ..., and Xinda Wang. ArXiv, 2025. 1 citations.

Not measured Topic Match

Abstract: The rapid advancement of pre-trained language models (PLMs) has demonstrated promising results for various code-related tasks. However, their effectiveness in detecting real-world vulnerabilities remains a critical challenge. % for the security community. While existing empirical studies evaluate PLMs for vulnerability detection (VD), their inadequate consideration in data preparation, evaluation setups, and experimental settings undermines the accuracy and comprehensiveness of evaluations. This paper introduces RevisitVD, an extensive evaluation of 17 PLMs spanning smaller code-specific PLMs and large-scale PLMs using newly constructed datasets. Specifically, we compare the performance of PLMs under both fine-tuning and prompt engineering, assess their effectiveness and generalizability across...

[151] [Safe4U: Identifying Unsound Safe Encapsulations of Unsafe Calls in Rust using LLMs](#)

Huan Li, ..., and Xin Xia. Proceedings of the ACM on Software Engineering, 2025. 0 citations.

Not measured Topic Match

Abstract: Rust is an emerging programming language that ensures safety through strict compile-time checks. A Rust function marked as unsafe indicates it has additional safety requirements (e.g., initialized, not null), known as contracts in the community. These unsafe functions can only be called within explicit unsafe blocks and the contracts must be guaranteed by the caller. To reuse and reduce unsafe code, the community recommends using safe encapsulation of unsafe calls (EUC) in practice. However, an EUC is unsound if any contract is not guaranteed and could lead to undefined behaviors in safe Rust, thus breaking Rust's safety promise. It is...

[152] [Large Language Models for Security Operations Centers: A Comprehensive Survey](#)

Ali Habibzadeh, ..., and Reza Ebrahimi Atani. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have emerged as powerful tools capable of understanding and generating human-like text, offering transformative potential across diverse domains. The Security Operations Center (SOC), responsible for safeguarding digital infrastructure, represents one of these domains. SOCs serve as the frontline of defense in cybersecurity, tasked with continuous monitoring, detection, and response to incidents. However, SOCs face persistent challenges such as high alert volumes, limited resources, high demand for experts with advanced knowledge, delayed response times, and difficulties in leveraging threat intelligence effectively. In this context, LLMs can offer promising solutions by automating log analysis, streamlining triage, improving detection...

[153] [Vul-R2: A Reasoning LLM for Automated Vulnerability Repair](#)

Xinjie Wen, ..., and Deheng Ye. ArXiv, 2025. 1 citations.

Not measured Topic Match

Abstract: The exponential increase in software vulnerabilities has created an urgent need for automatic vulnerability repair (AVR) solutions. Recent research has formulated AVR as a sequence generation problem and has leveraged large language models (LLMs) to address this problem. Typically, these approaches prompt or fine-tune LLMs to generate repairs for vulnerabilities directly. Although these methods show state-of-the-art performance, they face the following challenges: (1) Lack of high-quality, vulnerability-related reasoning data. Current approaches primarily rely on foundation models that mainly encode general programming knowledge. Without vulnerability-related reasoning data, they tend to fail to capture the diverse vulnerability repair patterns. (2) Hard to...

[154] [Frontier AI's Impact on the Cybersecurity Landscape](#)

Wenbo Guo, ..., and D. Song. ArXiv, 2025. 16 citations.

Not measured Topic Match

Abstract: The impact of frontier AI in cybersecurity is rapidly increasing. In this paper, we comprehensively analyze this trend through three distinct lenses: a quantitative benchmark analysis, a literature review, and an expert survey. We find that while AI is already widely used in attacks, its application in defense remains limited, especially in remediation and deployment. Aligned with these analyses, experts expect AI to continue favoring attackers over defenders, though the gap

will gradually narrow. These findings underscore the urgent need to mitigate frontier AI's risks while closely monitoring emerging capabilities. We provide concrete calls-to-action regarding: the construction of new cybersecurity...

[155] [jWith a Little Help from My \(LLM\) Friends: Enhancing Static Analysis with LLMs to Detect Software Vulnerabilities](#)

Amy Munson, ..., and Alvaro A. Cárdenas. 2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code), 2025. 1 citations.

Not measured Topic Match

Abstract: This paper explores the integration of Large Language Models (LLMs) with static analysis tools, specifically Semgrep, to enhance vulnerability detection in Java applications. Through a series of experiments, we evaluate the performance of various LLMs in triaging security weaknesses identified by Semgrep. We also study how LLMs perform across different types of vulnerabilities and assess the impact of various prompt engineering strategies. Our results reveal that while some LLM models reduce the accuracy of baseline results with static analysis, they show a consistent improvement with each new model released. In particular, o1-mini significantly outperformed others in our experiments in terms...

[156] [Can LLMs handle WebShell detection? Overcoming Detection Challenges with Behavioral Function-Aware Framework](#)

Feijiang Han, ..., and Yunhuai Liu. ArXiv, 2025. 3 citations.

Not measured Topic Match

Abstract: WebShell attacks, where malicious scripts are injected into web servers, pose a significant cybersecurity threat. Traditional ML and DL methods are often hampered by challenges such as the need for extensive training data, catastrophic forgetting, and poor generalization. Recently, Large Language Models have emerged as powerful alternatives for code-related tasks, but their potential in WebShell detection remains underexplored. In this paper, we make two contributions: (1) a comprehensive evaluation of seven LLMs, including GPT-4, LLaMA 3.1 70B, and Qwen 2.5 variants, benchmarked against traditional sequence- and graph-based methods using a dataset of 26.59K PHP scripts, and (2) the Behavioral Function-Aware...

[157] [VulnTeam: A Team Collaboration Framework for LLM-based Vulnerability Detection](#)

Jiayuan Li, ..., and Hongsong Zhu. 2025 International Joint Conference on Neural Networks (IJCNN), 2025. 0 citations.

Not measured Topic Match

Abstract: Software vulnerability detection is a critical challenge in cyber security. With the rise of deep learning and large language models (LLMs), numerous studies have applied these technologies to vulnerability detection. Existing approaches directly employ prompt engineering, chain-of-thought reasoning, and fine-tuning methods on LLMs, but achieve suboptimal results. To effectively leverage LLMs' powerful reasoning capabilities for vulnerability detection, we propose VulnTeam, a novel team collaboration framework for LLM vulnerability detection inspired by human expert team collaboration. Specifically, we introduce a dual-stage fine-tuning approach where expert models are first fine-tuned using low-rank adaptation to detect vulnerabilities related to different vulnerability syntactic features...

[158] [Guessing As A Service: Large Language Models Are Not Yet Ready For Vulnerability Detection](#)

Francesco Panebianco, ..., and Michele Carminati. Journal Not Provided, 2025. 2 citations.

Not measured Topic Match

No summary or abstract available

[159] [An Insight into Security Code Review with LLMs: Capabilities, Obstacles, and Influential Factors](#)

Jiaxin Yu, ..., and Yangxiao Cai. ArXiv, 2024. 10 citations.

Not measured Topic Match

Abstract: Security code review is a time-consuming and labor-intensive process typically requiring integration with automated security defect detection tools. However, existing security analysis tools struggle with poor generalization, high false positive rates, and coarse detection granularity. Large Language Models (LLMs) have been considered promising candidates for addressing those challenges. In this study, we conducted an empirical study to explore the potential of LLMs in detecting security defects during code review. Specifically, we evaluated the performance of six LLMs under five different prompts and compared them with state-of-the-art static analysis tools. We also performed linguistic and regression analyses for the best-performing LLM...

[160] [PortGPT: Towards Automated Backporting Using Large Language Models](#)

Zhaoyang Li, ..., and Dongliang Mu. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Patch backporting, the process of migrating mainline security patches to older branches, is an essential task in maintaining popular open-source projects (e.g., Linux kernel). However, manual backporting can be labor-intensive, while existing automated methods, which heavily rely on predefined syntax or semantic rules, often lack agility for complex patches. In this paper, we introduce PORTGPT, an LLM-agent for end-to-end automation of patch backporting in real-world scenarios. PORTGPT enhances an LLM with tools to access code on-demand, summarize Git history, and revise patches autonomously based on feedback (e.g., from compilers), hence, simulating human-like reasoning and verification. PORTGPT achieved an 89.15% success...

[161] [Write Your Own CodeChecker: An Automated Test-Driven Checker Development Approach with LLMs](#)

Jun Liu, ..., and Jian Zhang. ArXiv, 2024. 1 citations.

Not measured Topic Match

Abstract: With the rising demand for code quality assurance, developers are not only utilizing existing static code checkers but also seeking custom checkers to satisfy their specific needs. Nowadays, various code-checking frameworks provide extensive checker customization interfaces to meet this need. However, both the abstract checking logic and the complex API usage of large-scale checker frameworks make this task challenging. To this end, automated code checker generation is anticipated to ease the burden of checker development. In this paper, we propose AutoChecker, an innovative LLM-powered approach that can write code checkers automatically based on only a rule description and a test...

[162] [Enhancing Vulnerability Detection via Inter-procedural Semantic Completion](#)

Bozhi Wu, ..., and Shang-Wei Lin. Proceedings of the ACM on Software Engineering, 2025. 1 citations.

Not measured Topic Match

Abstract: Inspired by advances in deep learning, numerous learning-based approaches for vulnerability detection have emerged, primarily operating at the function level for scalability. However, this design choice has a critical limitation: many vulnerabilities span multiple functions, causing function-level approaches to lose the semantics of called functions and fail to capture true vulnerability patterns. To address this issue, we propose VulnSC, a novel framework designed to enhance learning-based approaches by complementing inter-procedural semantics. VulnSC retrieves the source code of called functions for datasets and leverages large language models (LLMs) with well-designed prompts to generate summaries for these functions. The datasets, enhanced with...

[163] [Measuring and Augmenting Large Language Models for Solving Capture-the-Flag Challenges](#)

Zimo Ji, ..., and Shuai Wang. ArXiv, 2025. 2 citations.

Not measured Topic Match

Abstract: Capture-the-Flag (CTF) competitions are crucial for cybersecurity education and training. As large language models (LLMs) evolve, there is increasing interest in their ability to automate CTF challenge solving. For example, DARPA has organized the AIXCC competition since 2023 to advance AI-powered automated offense and defense. However, this demands a combination of multiple abilities, from knowledge to reasoning and further to actions. In this paper, we highlight the importance of technical knowledge in solving CTF problems and deliberately construct a focused benchmark, CTFKnow, with 3,992 questions to measure LLMs' performance in this core aspect. Our study offers a focused and innovative measurement...

[164] [Adaptive Plan-Execute Framework for Smart Contract Security Auditing](#)

Zhiyuan Wei, ..., and Zixiao Zhao. ArXiv, 2025. 2 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have shown great promise in code analysis and auditing; however, they still struggle with hallucinations and limited context-aware reasoning. We introduce SmartAuditFlow, a novel Plan-Execute framework that enhances smart contract security analysis through dynamic audit planning and structured execution. Unlike conventional LLM-based auditing approaches that follow fixed workflows and predefined steps, SmartAuditFlow dynamically generates and refines audit plans based on the unique characteristics of each smart contract. It continuously adjusts its auditing strategy in response to intermediate LLM outputs and newly detected vulnerabilities, ensuring a more adaptive and precise security assessment. The framework then executes these...